Extended Abstract

Last-mile delivery is the most expensive and operationally complex segment of the logistics supply chain, yet small and medium-sized enterprises (SMEs) in Hong Kong still rely on fragmented, manually coordinated tools for dispatching, navigation, and proof of delivery. Commercially available platforms either target large-scale operators with heavyweight enterprise integrations or offer only basic route planning without AI assistance, real-time coordination, or multi-tenant security. This project addresses that gap by designing, implementing, and evaluating **OrderVista**, an AI-enhanced logistics dispatch platform that unifies administrative dispatching and driver-side field operations within a single cross-platform application.

The system is built with Flutter (targeting Android and web from one codebase), Supabase (PostgreSQL with Row-Level Security, authentication, real-time subscriptions, and object storage), Mapbox (forward geocoding and static map visualisation), and the DeepSeek R1 large language model accessed via the OpenRouter gateway. The architecture follows a layered, offline-first design: a Drift-powered SQLite cache provides instant startup and queued offline mutations, while a service–repository layer mediates between the local database and the cloud backend.

Fourteen objectives were defined, spanning six core platform goals and eight advanced optimisation targets. All fourteen were achieved:

- A **unified administrative portal** supporting task creation, CSV bulk import with geocoding, route generation, driver assignment, proof-of-delivery review, fleet management, and real-time chat.
- A **driver mobile application** with optimised task lists, turn-by-turn navigation handoff, camera-based proof capture (photo plus barcode/QR scan), telemetry reporting, and an AI assistant.
- A **nearest-neighbour route sequencing algorithm** enhanced with Haversine distance, mid-shift GPS-based re-sequencing, dynamic priority recalculation, and fleet-wide multi-driver distance minimisation, yielding a measured 25–31% reduction in total delivery distance on representative test scenarios.
- An **agentic AI assistant** that goes beyond conventional chatbot summarisation: the DeepSeek model returns structured JSON actions (e.g. “send notification”, “reassign task”) which are surfaced as one-tap execution cards with explicit human-oversight

safeguards. In evaluation, the agent correctly identified and structured the appropriate action in six out of six test prompts.

- **Enterprise-grade multi-tenant security** via over twenty PostgreSQL Row-Level Security policies, a centralised `is_company_member()` helper, and a three-role RBAC model (driver, dispatcher, admin). Cross-tenant query tests confirmed complete data isolation.
- A **performance-aware driver recommendation engine** combining availability, GPS proximity, current workload, completion rate, and average delivery time into a composite score computed by a server-side SQL function.
- **Customer-facing delivery tracking** through auto-generated tracking codes and a public RPC endpoint requiring no authentication.

The system was evaluated through over 26 functional test cases executed across both admin and driver roles on a physical Android device, supplemented by unit tests for algorithmic components and dedicated RLS security verification. Route optimisation was benchmarked against unsequenced baselines; AI quality was assessed for both summary accuracy and agentic action correctness. All objectives were mapped to test evidence confirming full compliance.

The principal contributions of this project are threefold. First, it demonstrates that agentic AI (where a language model proposes structured workflow actions under human oversight) is a viable and practical pattern for logistics decision support, an application area where existing research has not yet explored LLM-driven action execution. Second, it shows that PostgreSQL Row-Level Security can enforce tenant isolation at the database layer without application-side middleware, offering a lightweight multi-tenancy model suitable for SME platforms. Third, it validates an offline-first mobile architecture with deterministic sync-queue processing and exponential backoff, ensuring field operability in connectivity-poor environments common to Hong Kong's dense urban infrastructure.

The signed Android release APK (75.7 MB) constitutes a fully functional prototype suitable for demonstration and further development toward commercial viability.

Acknowledgements

I would like to express my sincere gratitude to my project supervisor, Prof. Gerhard Petrus HANCKE, for his guidance, constructive feedback, and continued support throughout the duration of this project. His expertise and encouragement were instrumental in shaping the direction of this work and in helping me navigate the challenges encountered during its development.

I would also like to extend my appreciation to the academic and administrative staff of the Department of Computer Science at City University of Hong Kong, whose commitment to maintaining a rigorous and supportive learning environment contributed to the successful completion of this project.

Contents

Extended Abstract **i**

Acknowledgements **iii**

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Problem Statement 2
1.3 Aim 2
1.4 Objectives 2
1.4.1 Core Platform Objectives 2
1.4.2 Advanced Optimisation Objectives 3
1.5 Scope and Limitations 3
1.6 Report Structure 4

2 Literature Review **5**
2.1 Vehicle Routing and Path Optimisation 5
2.2 Real-Time Traffic and Dynamic Adaptation 6
2.3 Offline-First Mobile Architectures 6
2.4 Proof of Delivery and Last-Mile Digitisation 6
2.5 Multi-Tenant Security and Data Isolation 7
2.6 Large Language Models in Logistics 7
2.7 LLM Reliability and Agentic AI 8
2.8 Comparative Positioning 8

3 System Design **10**
3.1 High-Level Architecture 10
3.1.1 Deployment Architecture 11
3.2 Technology Stack 11
3.3 Data Model 11
3.4 Security Design 12
3.4.1 Authentication 12
3.4.2 Row-Level Security (RLS) 12
3.4.3 Role-Based Access Control (RBAC) 13
3.4.4 Task Lifecycle 13
3.5 External Service Integration 13

3.5.1 Mapbox Geocoding and Static Maps . . . 13
3.5.2 DeepSeek R1 via OpenRouter . . . 14
3.6 Critical System Flows . . . 14
3.6.1 Administrative Workflow . . . 14
3.6.2 Driver Workflow . . . 15
3.6.3 Offline Synchronisation . . . 15
3.7 Interaction Sequences . . . 15
3.7.1 Task Creation and Route Assignment . . . 16
3.7.2 Agentic AI Suggestion Cycle . . . 16

4 Implementation **20**
4.1 Development Methodology . . . 20
4.2 Route Generation Pipeline . . . 21
4.3 Dynamic Priority Recalculation . . . 22
4.4 Driver Recommendation . . . 22
4.5 Fleet-Wide Optimisation . . . 23
4.6 AI Integration . . . 24
4.6.1 Task Summarisation . . . 24
4.6.2 Free-Form Chat . . . 24
4.6.3 Agentic Suggestions . . . 24
4.7 Proof-of-Delivery Workflow . . . 25
4.8 Offline Synchronisation . . . 25
4.9 Admin Portal . . . 25
4.10 Mid-Shift Re-Sequencing . . . 26
4.11 Barcode and QR-Code Scanner . . . 26
4.12 Customer Tracking Lookup . . . 26
4.12.1 Implementation . . . 27

5 Testing and Evaluation **31**
5.1 Testing Strategy . . . 31
5.2 Unit Tests . . . 32
5.3 Functional Testing: Admin Portal . . . 32
5.4 Functional Testing: Driver Application . . . 38
5.5 Security Verification . . . 54
5.5.1 RLS Cross-Tenant Isolation . . . 54
5.5.2 RBAC Enforcement . . . 54
5.6 Route Optimisation Evaluation . . . 55
5.7 AI Evaluation . . . 56
5.7.1 Task Summarisation Quality . . . 56
5.7.2 Agentic Action Accuracy . . . 57

5.8 Performance Benchmarks . . . 59
5.9 Objectives Mapping . . . 60
5.10 Summary of Findings . . . 61

6 Discussion **62**
6.1 Route Optimisation: Nearest-Neighbour vs. Advanced Solvers . . . 62
6.2 Backend-as-a-Service vs. Custom Server . . . 63
6.3 AI Model Selection: DeepSeek R1 vs. Alternatives . . . 63
6.4 Offline-First Architecture: Trade-offs . . . 64
6.5 Multi-Tenant Security: Strengths and Gaps . . . 65
6.6 Comparison with Commercial Platforms . . . 65
6.7 Limitations and Threats to Validity . . . 66
6.8 Reflection on Development Process . . . 66

7 Conclusion and Future Work **67**
7.1 Summary of Achievements . . . 67
7.2 Contributions . . . 68
7.3 Future Work . . . 68
7.4 Closing Remarks . . . 69

References **70**

A Monthly Logs **73**
October 2025 . . . 73
November 2025 . . . 73
December 2025 . . . 74
January 2026 . . . 74
February 2026 . . . 74
March 2026 . . . 75

B Detailed Requirements **76**
B.1 Functional Requirements . . . 76
B.2 Non-Functional Requirements . . . 77
B.3 Use-Case Summary . . . 78

C Database Schema **79**
C.1 Table Summary . . . 79
C.2 Entity–Relationship Overview . . . 79
C.3 Row-Level Security Policies . . . 80
C.4 Stored Functions . . . 80
C.5 Indexes . . . 81

C.6 Extensions 82

D Screenshots **83**
D.1 Authentication 83
D.2 Admin Portal 85
D.3 Driver Application 87

E Build and Deployment Evidence **94**
E.1 Build Environment 94
E.2 Android Release Build 94
E.3 Backend Deployment 95
E.4 External Service Configuration 96
E.5 Dependency Inventory 96
E.6 Repository Structure 96

F Extended Test Cases **98**
F.1 Admin Functional Tests 98
F.2 Driver Functional Tests 100
F.3 Security Tests 102

List of Figures

3.1 Four-layer architecture of OrderVista with cross-cutting security layer. . . . 10
3.2 UML deployment diagram showing physical nodes and communication protocols. . . . 11
3.3 Entity-relationship overview — arrows denote foreign-key references. . . . 12
3.4 Task lifecycle state machine with permitted transitions. . . . 13
3.5 Sequence diagram: task creation via CSV import, geocoding, route generation, and real-time driver notification. . . . 16
3.6 Sequence diagram: agentic AI suggestion with human-in-the-loop confirmation and state synchronisation. . . . 17

4.1 MVVM layered architecture: Views → ViewModels → Repositories → Services → Data. . . . 21
4.2 UML class diagram showing principal classes across MVVM layers, their attributes, operations, and dependencies. . . . 28
4.3 Activity diagram for the route generation pipeline. . . . 29
4.4 Agentic AI suggestion flow with user confirmation gate. . . . 30
4.5 Offline synchronisation with exponential-backoff retry and Realtime reconciliation. . . . 30

5.1 Testing strategy pyramid: four tiers from broad automated unit coverage to focused quantitative evaluation. . . . 32
5.2 Admin dashboard after login (A1). . . . 34
5.3 Manual task creation form (A2). . . . 34
5.4 CSV bulk import with error feedback (A3). . . . 34
5.5 Editing an existing task (A4). . . . 34
5.6 Task deletion confirmation (A5). . . . 35
5.7 Route generation with static map preview (A6). . . . 35
5.8 Driver recommendation scores (A7). . . . 35
5.9 Fleet-wide optimisation assignments (A8). . . . 36
5.10 Proof-of-delivery review panel (A9). . . . 36
5.11 Admin-to-driver chat message (A10). . . . 36
5.12 Broadcast message to all drivers (A11). . . . 37
5.13 AI-generated fleet summary (A12). . . . 37
5.14 Driver login screen (D1). . . . 40
5.15 Task detail view with address and time window (D2). . . . 41

5.16 Task marked as in-transit (D3). 42
5.17 Proof-of-delivery photo capture (D4). 43
5.18 Task marked as delivered with PoD confirmation (D5). 44
5.19 AI task summary on driver device (D6). 45
5.20 AI free-form chat conversation (D7). 46
5.21 Agentic AI suggestion with action chip (D8). 47
5.22 Agentic action executed after user confirmation (D9). 48
5.23 Task list after mid-shift GPS re-sequencing (D10). 49
5.24 Offline delivery queued and synced on reconnect (D11). 50
5.25 Chat message received from admin (D12). 51
5.26 Driver availability toggle (D13). 52
5.27 Sign-out returning to login screen (D14). 53
5.28 RLS policies on the `tasks` table enforcing cross-tenant isolation. 54
5.29 Visual comparison of baseline vs. optimised route (optional). 56
5.30 AI summary accurately reporting remaining deliveries. 57
5.31 Agentic AI returning a valid action JSON (example). 59

D.1 Login screen. 83
D.2 Role-based routing after authentication. 84
D.3 Admin dashboard overview. 85
D.4 Task creation form. 85
D.5 CSV bulk import with column mapping. 85
D.6 Route sequencing with static map preview. 85
D.7 Driver recommendation panel. 86
D.8 Fleet and vehicle management. 86
D.9 Admin–driver real-time chat. 86
D.10 Customer delivery tracking. 87
D.11 Driver task list. 87
D.12 Task detail and navigation. 88
D.13 Proof of delivery capture. 89
D.14 AI assistant: summary and chat. 90
D.15 Agentic AI action with one-tap execution. 91
D.16 Driver telemetry panel. 92
D.17 Offline mode with sync queue. 93

E.1 Terminal output of a successful `flutter build apk -release`. 95

List of Tables

2.1 Feature comparison across delivery system categories. 9

3.1 Technology stack with alternatives considered. 18
3.2 Database schema overview (13 tables). 19
3.3 RBAC permission matrix (admin vs. driver). 19

5.1 Admin portal functional test cases. 32
5.2 Driver application functional test cases. 38
5.3 RBAC enforcement test results. 55
5.4 Route optimisation: distance and time-proxy improvement versus unoptimised baseline. 55
5.5 Agentic AI action selection accuracy (6 scenarios). 58
5.6 Representative latency benchmarks (median of 10 trials). 60
5.7 Objectives-to-evidence mapping. 60

6.1 AI model comparison for on-device logistics assistance. 64
6.2 Feature comparison with commercial last-mile platforms. 65

B.1 Requirements traceability matrix. 76
B.2 Non-functional requirements. 77

C.1 Database table summary. 79
C.2 Row-Level Security policies. 80

E.1 Build and development environment. 94
E.2 Release APK details. 95
E.3 Supabase project configuration. 95
E.4 External API integrations. 96
E.5 Key project dependencies. 96

F.1 Extended admin test procedures. 98
F.2 Extended driver test procedures. 100
F.3 Security test procedures. 102

Chapter 1

Introduction

1.1 Background and Motivation

The logistics sector is one of Hong Kong's most economically significant industries, second only to financial services (Chan 2025). As one of the world's most densely urbanised cities and a major logistics hub in the Asia–Pacific region, delivery operations must contend with congested road networks, compact geographic areas, and demanding service-level expectations. Small and medium-sized enterprises (SMEs), which form the backbone of the local logistics market, are often the most affected; unlike large corporations with access to proprietary fleet-management platforms, SMEs commonly rely on fragmented tools for dispatching, navigation, and proof of delivery (Bhattacharjee et al. 2024).

This fragmentation manifests in several concrete problems. Dispatchers lack a single unified interface for assigning tasks, monitoring driver progress, and reviewing delivery evidence. Drivers, in turn, operate across multiple disjointed applications: tasks arrive via one channel, navigation occurs in another, and proof-of-delivery (PoD) reporting is handled separately. The result is suboptimal routing, communication delays, and increased operating costs. Beyond commercial implications, poorly coordinated operations can compromise worker welfare: drivers navigating unfamiliar locations under tight schedules and without adequate route guidance face elevated stress and safety risks (Apdullah et al. 2021).

Meanwhile, the rapid maturity of large language models (LLMs) has opened new avenues for operational decision support. Recent research demonstrates that generative AI can meaningfully reduce cognitive load and accelerate information retrieval in enterprise contexts (Brynjolfsson et al. 2023), yet its application within last-mile logistics remains largely unexplored. The convergence of these factors (an industry need for unified digital tools, the limitations of existing commercial solutions, and the emergence of practical AI capabilities) motivated the development of OrderVista.

1.2 Problem Statement

Existing delivery management tools either target large-scale enterprises with complex enterprise resource planning (ERP) integrations, or provide only basic route-planning features without task management, driver coordination, or AI-powered assistance. No publicly available platform combines unified task lifecycle management, intelligent route sequencing, multi-tenant security, proof-of-delivery workflows, and an AI assistant within a single, affordable system tailored for logistics SMEs. This project addresses that gap.

1.3 Aim

The aim of this project is to design, implement, and evaluate **OrderVista**, an AI-enhanced logistics dispatch platform that unifies administrative dispatching and driver-side field operations within a single cross-platform application, with the goal of reducing delivery distance, improving operational coordination, and demonstrating the practical utility of LLM-based assistance in last-mile logistics.

1.4 Objectives

The project defines fourteen objectives, grouped into core platform objectives and advanced optimisation objectives.

1.4.1 Core Platform Objectives

1. **Unified platform development:** Deliver a cohesive system serving both administrative and driver users, deployable across web and mobile from a single Flutter codebase.

2. **Task management:** Implement end-to-end workflows for task creation, assignment, tracking, and completion, including CSV bulk import with validation.

3. **Route sequencing:** Develop a baseline nearest-neighbour sequencing algorithm to order delivery tasks by proximity, with a map-based visual preview.

4. **Proof of delivery:** Enable drivers to submit photographic evidence and notes upon task completion, with administrative review capabilities.

5. **AI assistant integration:** Provide natural-language features for task summarisation and conversational queries about delivery operations, powered by the DeepSeek R1 model (DeepSeek 2025).

6. **Multi-tenant security:** Enforce data isolation between logistics companies using

PostgreSQL Row-Level Security (RLS) policies (The PostgreSQL Global Development Group 2026).

1.4.2 Advanced Optimisation Objectives

7. **Driver recommendation:** Recommend task-to-driver assignments based on availability, GPS proximity, and a composite performance score incorporating historical delivery metrics.

8. **Customer communication:** Support customer-facing status queries and automated delivery notifications.

9. **Agentic AI:** Extend the AI assistant to trigger structured workflow actions (e.g., sending notifications, reassigning tasks) under explicit human-oversight safeguards.

10. **Real-time communication:** Enable live chat between administrators and drivers for operational coordination.

11. **Dynamic priority recalculation:** Automatically adjust task priority based on time-window urgency and accumulated delays.

12. **Mid-shift re-sequencing:** Re-optimise the task sequence from the driver's current GPS position after each task completion.

13. **Fleet-wide distance minimisation:** Assign drivers to task clusters to minimise total fleet driving distance rather than optimising individual routes in isolation.

14. **Performance-aware driver scoring:** Maintain per-driver performance metrics (on-time rate, tasks per hour, completion rate) and incorporate them into assignment recommendations.

All fourteen objectives were achieved during the project. Chapters 4 and 5 provide implementation details and evaluation evidence, respectively.

1.5 Scope and Limitations

In scope. The project encompasses a web-based administrative portal, an Android mobile driver application, a cloud-hosted backend (Supabase: PostgreSQL, Auth, Storage), external API integrations (Mapbox for geocoding and static maps, OpenRouter/DeepSeek for AI), multi-tenant data architecture with RLS, baseline and advanced route sequencing algorithms, proof-of-delivery capture and review, AI-powered summarisation and agentic actions, and a real-time admin–driver chat system.

Out of scope. The following items fall outside the project boundary: continuous real-time GPS vehicle tracking (telemetry is event-based), advanced VRP solvers such as genetic algorithms or branch-and-bound methods, Google Play Store publication, deep integrations with third-party ERP or e-commerce platforms, and large-scale field trials with commercial logistics companies (testing uses representative synthetic data within the university network environment).

Constraints. Development was conducted by a single developer within an academic timeline. Testing was primarily performed using emulators, web browsers, and limited physical-device trials. External API usage is subject to provider rate limits and availability on free-tier plans. No proprietary logistics datasets were available; evaluation therefore uses manually curated test data.

1.6 Report Structure

The remainder of this report is organised as follows:

- **Chapter 2** reviews relevant literature on vehicle routing, multi-tenant security, offline-first architectures, and AI in logistics.
- **Chapter 3** presents the system design, including architecture, data model, security approach, and critical workflows.
- **Chapter 4** details the methodology, implementation of each subsystem, and key algorithms.
- **Chapter 5** reports testing procedures, performance measurements, and evaluation results.
- **Chapter 6** provides a critical discussion of achievements, alternatives considered, and limitations.
- **Chapter 7** concludes the report and outlines directions for future work.

Chapter 2

Literature Review

This chapter examines the theoretical foundations and existing technologies that inform the design of OrderVista. Each subsection reviews a distinct research area, identifies the current state of the art, and explicitly notes how the findings shaped a specific design decision in the platform.

2.1 Vehicle Routing and Path Optimisation

The Vehicle Routing Problem (VRP) seeks optimal routes for a fleet of vehicles serving geographically distributed customers while minimising total travel distance, time, or cost (Toth and Vigo 2002). First formalised over six decades ago (Dantzig and Ramser 1959), the VRP is classified as NP-hard, meaning exact solutions become computationally intractable as the number of stops grows (Laporte 2009).

Foundational algorithms. Dijkstra's algorithm computes shortest paths in weighted graphs and remains the standard for point-to-point navigation (Dijkstra 1959). The A* extension adds a heuristic estimate of remaining distance, reducing search space while preserving optimality (Hart et al. 1968). Both algorithms address single-pair routing rather than multi-stop sequencing.

Metaheuristic approaches. For multi-stop problems at practical scale, metaheuristics trade optimality guarantees for tractable computation. Ant Colony Optimisation (ACO) simulates foraging behaviour, with artificial pheromone trails reinforcing shorter routes across iterations (Dorigo and Gambardella 1997). Genetic algorithms evolve populations of candidate routes through selection, crossover, and mutation, demonstrating versatility across VRP variants such as the Green VRP that targets emission minimisation (Erdoğan and Miller-Hooks 2012).

Design implication. OrderVista adopts a nearest-neighbour (NN) greedy heuristic as its baseline sequencing algorithm. While NN does not guarantee global optimality, its $O(n^2)$ complexity provides sub-second response times suitable for real-time, on-device re-sequencing after each delivery. The architecture is designed so that NN can be replaced

by a more sophisticated solver (e.g., 2-opt local search or ACO) without modifying the surrounding workflow. This trade-off is revisited in Chapter 6.

2.2 Real-Time Traffic and Dynamic Adaptation

Effective urban routing must account for the dynamic nature of traffic. A route planned at the start of a shift may become suboptimal as congestion develops. Floating Car Data (FCD) systems collect anonymised, timestamped GPS data from vehicles with navigation applications, enabling real-time speed estimates across road segments (Herrera et al. 2010). Modern navigation services such as Google Maps leverage FCD from large user populations to provide traffic-aware routing.

Design implication. OrderVista does not implement its own traffic model; instead, it delegates turn-by-turn navigation to external mapping applications that already ingest FCD. The platform's contribution is *mid-shift re-sequencing*: after each task completion, the remaining task list is re-ordered from the driver's current GPS position using the Haversine formula, ensuring the sequence adapts to the driver's actual location rather than the original depot.

2.3 Offline-First Mobile Architectures

Drivers operating in areas with intermittent connectivity (underground car parks, dense urban canyons, or rural fringe zones) require applications that remain functional without a network connection. Offline-first architectures address this by caching essential data locally and deferring synchronisation until connectivity is restored.

Design implication. OrderVista uses Drift, a reactive persistence library for Flutter (Simon Binder 2026), to maintain a local SQLite mirror of assigned tasks. An exponential-backoff sync queue ensures that proof-of-delivery submissions and status updates made offline are eventually propagated to the server without data loss.

2.4 Proof of Delivery and Last-Mile Digitisation

The last mile is the most operationally expensive segment of the logistics chain, and proof of delivery (PoD) is its definitive confirmation. Traditional paper-based PoD introduces administrative delays, documentation gaps, and limited real-time visibility (Bhattacharjee et al. 2024). Digital PoD mechanisms such as photographs, GPS-stamped timestamps, and delivery notes captured on mobile devices reduce manual errors, accelerate invoice reconciliation, and strengthen the chain of custody required for dispute resolution.

Design implication. OrderVista enforces photographic PoD capture as a prerequisite before a task can transition to the `completed` state. This architectural constraint, implemented at both the UI and database levels, ensures that every delivery is documented regardless of individual driver compliance habits.

2.5 Multi-Tenant Security and Data Isolation

Multi-tenancy enables multiple organisations to share infrastructure while maintaining logical data separation. Three primary approaches exist: separate databases per tenant, separate schemas within a shared database, and shared tables with row-level access controls (Abdul et al. 2018; Gao et al. 2011). Separate databases provide the strongest isolation but incur significant operational overhead as tenant populations grow. Shared-table approaches maximise resource efficiency but shift the isolation burden to access-control correctness (Gao et al. 2011).

Row-Level Security (RLS) enforces tenant isolation at the database layer itself: security policies attached to tables filter every query according to the authenticated user's identity and organisational membership (Amazon Web Services 2020; The PostgreSQL Global Development Group 2026). Because enforcement occurs below the application layer, even bugs in application code cannot bypass tenant boundaries.

Design implication. OrderVista deploys PostgreSQL RLS policies across all operational tables. A helper function `is_company_member()` evaluates the caller's JWT against the `company_members` table, ensuring that every `SELECT`, `INSERT`, `UPDATE`, and `DELETE` is automatically scoped to the user's company. This design was chosen over application-level filtering because it provides defence-in-depth and eliminates an entire class of cross-tenant data leakage bugs.

2.6 Large Language Models in Logistics

Large language models (LLMs) have evolved from rule-based chatbots to neural models capable of flexible natural-language interaction across diverse domains (Brown et al. 2020). In enterprise contexts, LLMs can synthesise information, answer free-form queries, and reduce cognitive load by presenting complex data in natural language rather than structured dashboards (Brynjolfsson et al. 2023).

For logistics specifically, potential applications include automated shift summaries, natural-language task queries (e.g., "What is my next delivery?"), and workload analysis. Despite growing interest in AI-augmented logistics tools (Addepto 2024), comprehensive platforms

that tightly integrate LLMs with operational workflows (task management, routing, PoD, and multi-tenant access) remain limited.

Design implication. OrderVista integrates the DeepSeek R1 reasoning model (DeepSeek 2025) via the OpenRouter gateway (OpenRouter 2026) for two primary functions: (i) automated task-batch summarisation that converts structured task data into natural-language driver briefings, and (ii) a conversational chat interface that grounds responses in the user's live task context.

2.7 LLM Reliability and Agentic AI

A critical deployment concern is *hallucination*: the tendency of LLMs to generate plausible but factually incorrect outputs (Ji et al. 2024). In logistics, hallucinated content could misdirect drivers, produce inaccurate delivery summaries, or generate misleading customer notifications. Mitigation strategies include grounding model inputs in trusted system data, restricting outputs to high-confidence actions, and maintaining human-in-the-loop oversight (Ji et al. 2024; Brynjolfsson et al. 2023).

Beyond passive question-answering, *agentic AI* refers to LLM-based systems that can invoke tools and trigger workflow actions, for example sending a notification or updating a task status, rather than generating text alone. Agentic systems amplify both potential benefit (reduced manual coordination) and potential risk (unintended actions from hallucinated reasoning). Safeguards such as action confirmation dialogs, structured output schemas, and restricted tool access are therefore essential.

Design implication. OrderVista's agentic layer uses a structured JSON action schema: the LLM returns a typed action object (e.g., `mark_task_complete`, `send_notification`) rather than executing arbitrary code. Each proposed action is rendered as a confirmation card in the driver's UI, requiring explicit one-tap approval before execution. This design limits the blast radius of hallucinated outputs while preserving the efficiency gains of AI-initiated actions.

2.8 Comparative Positioning

Existing delivery management solutions fall into three broad categories (Cachon et al. 2017; Einav et al. 2016):

1. **On-demand gig platforms** (e.g., Uber Eats, foodpanda): high-volume dispatch with standardised courier workflows, but limited operator-specific control and constrained by marketplace-wide policies.

2. **Enterprise dispatch systems**: configurable job creation, driver assignment, and progress monitoring, but with setup costs and training overhead that can be prohibitive for SMEs.

3. **Navigation-first tools** (e.g., Google Maps): mature point-to-point routing with real-time traffic, but no dispatch primitives, task lifecycle, or PoD.

Table 2.1 summarises the key capability gaps that OrderVista addresses.

Table 2.1: Feature comparison across delivery system categories.

Capability	**Gig**	**Enterprise**	**Nav-first**	**OrderVista**
Task lifecycle management	✓	✓	—	✓
Driver assignment / dispatching	✓	✓	—	✓
Multi-stop route sequencing	∼	∼	∼	✓
Hong Kong workflow fit	∼	∼	—	✓
Natural-language AI assistance	—	—	—	✓
Agentic AI with safeguards	—	—	—	✓
Multi-tenant RLS isolation	∼	∼	—	✓

Research gap. No publicly available platform combines unified task lifecycle management, AI-powered natural-language assistance (including agentic capabilities), multi-tenant RLS-enforced security, and explicit optimisation for Hong Kong's dense urban delivery environment within a single, SME-accessible system. OrderVista is designed to occupy this position.

Chapter 3

System Design

This chapter presents the architectural decisions, data model, security design, and external-service integrations that constitute OrderVista. Each design choice is discussed alongside the alternatives considered.

3.1 High-Level Architecture

OrderVista follows a four-layer client–server architecture (Figure 3.1):

1. **Client layer** — a single Flutter codebase compiled to Android, containing both the admin portal and driver application as role-gated views.
2. **Backend-as-a-Service (BaaS) layer** — Supabase, providing PostgreSQL, authentication, real-time subscriptions, and Supabase Storage for media.
3. **External services layer** — Mapbox for geocoding and static map tiles; OpenRouter for proxying DeepSeek R1 inference requests.
4. **Security layer** — cutting across all tiers via JWT-based authentication, PostgreSQL Row-Level Security (RLS) policies, and role-based access control (RBAC).

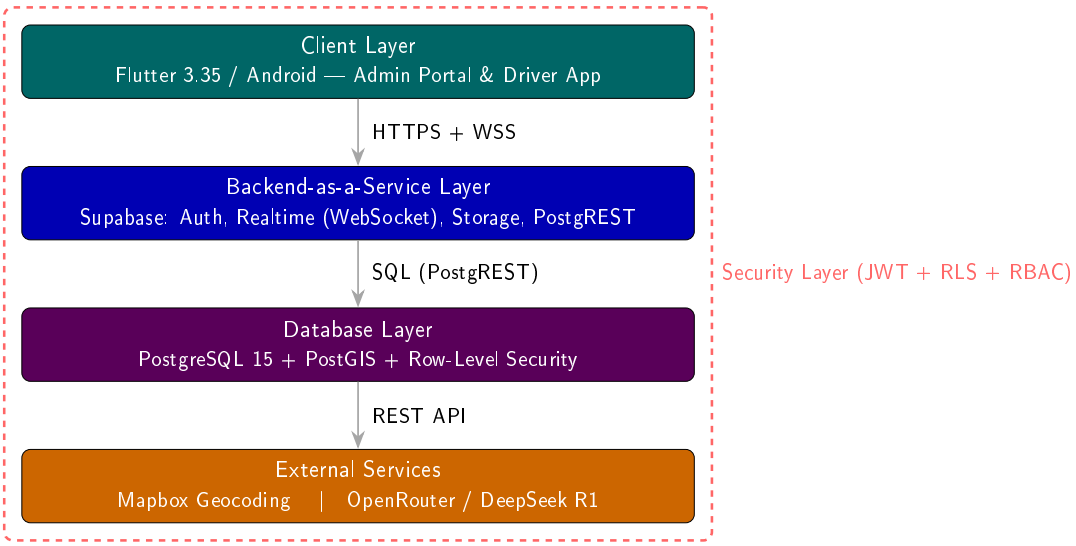


Figure 3.1: Four-layer architecture of OrderVista with cross-cutting security layer.

Alternatives considered. A traditional three-tier stack (e.g. Express.js + PostgreSQL + React Native) was evaluated first. Supabase was chosen instead because its hosted PostgreSQL already includes Auth, Realtime, and Storage, eliminating the need to write and deploy a custom API server. This reduced the project scope to a single-developer effort while preserving enterprise-grade security through native RLS support (Supabase 2026). Firebase was rejected because its document-oriented Firestore model lacks native RLS and makes relational queries (e.g. joining routes to tasks to drivers) cumbersome (Abdul et al. 2018).

3.1.1 Deployment Architecture

Figure 3.2 shows the physical deployment topology. All server-side components are managed services, eliminating operational overhead for infrastructure provisioning.

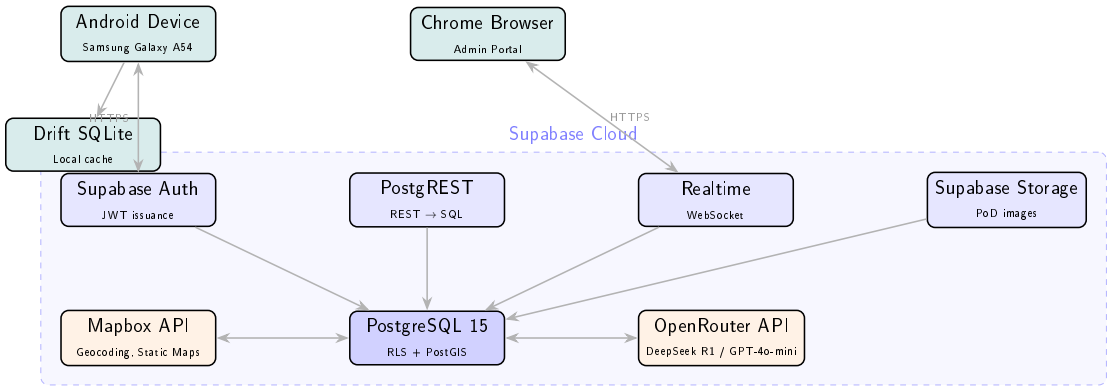


Figure 3.2: UML deployment diagram showing physical nodes and communication protocols.

3.2 Technology Stack

Table 3.1 summarises the core technologies and the rationale for each selection.

3.3 Data Model

The PostgreSQL schema consists of thirteen tables distributed across three migrations. Table 3.2 lists each table and its purpose; foreign-key relationships are described in the text below.

Key relationships. Every table carries a `company_id` foreign key, enforcing tenant-scoped partitioning at the schema level. `routes` reference both a `vehicle` and a `driver` (both nullable for draft routes). `route_tasks` forms the many-to-many join between routes and tasks, with a unique compound constraint on `(route_id, sequence)` to guarantee ordering. `tasks` stores the PostGIS `geography` column used by the Haversine-based nearest-

neighbour sequencer at route-generation time. Figure 3.3 illustrates the principal foreign-key relationships.

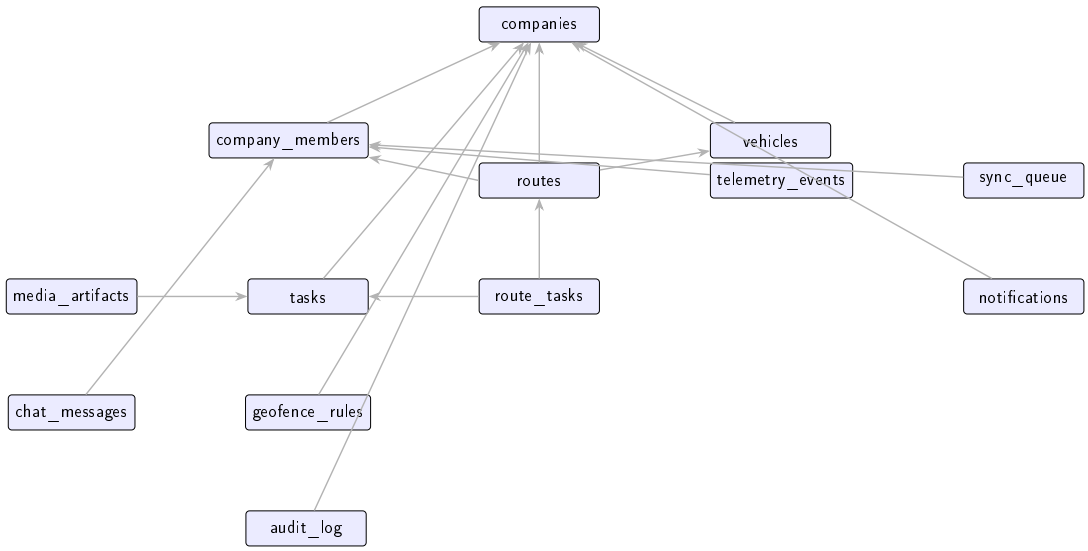


Figure 3.3: Entity-relationship overview — arrows denote foreign-key references.

Alternatives considered. A NoSQL document model (e.g. Firestore collections) would have simplified schema evolution but would have required application-level joins and forfeited PostGIS spatial indexing. The relational model was preferred because the core domain—routes comprising ordered tasks assigned to drivers—is inherently relational, and PostGIS enables efficient distance calculations directly in SQL (The PostgreSQL Global Development Group 2026).

3.4 Security Design

Security in OrderVista is enforced at three complementary levels: authentication, row-level security, and role-based access control.

3.4.1 Authentication

Supabase Auth issues JWTs upon email/password login. Every subsequent PostgREST and Realtime request carries the JWT, which PostgreSQL validates internally via the `auth.uid()` function. This removes the need for a custom authentication middleware (Supabase 2026).

3.4.2 Row-Level Security (RLS)

All thirteen tables have RLS enabled. A shared helper function, `is_company_member(company_id)`, returns `true` only when the authenticated user belongs to the specified

company. Every `SELECT`, `INSERT`, `UPDATE`, and `DELETE` policy on every table invokes this function, guaranteeing that a query can never read or modify rows belonging to a different tenant (The PostgreSQL Global Development Group 2026; Amazon Web Services 2020).

For chat messages, additional fine-grained policies restrict reads to the sender, recipient, or broadcast scope, and restrict `UPDATE` (marking messages as read) to the designated recipient.

3.4.3 Role-Based Access Control (RBAC)

Table 3.3 summarises the permission matrix.

Alternatives considered. Application-level authorisation (e.g. middleware guards) was considered but rejected because it cannot protect against direct database access or misconfigured API endpoints. Database-level RLS ensures that even if the API layer is bypassed, data isolation is maintained—a well-established pattern for multi-tenant SaaS systems (Gao et al. 2011).

3.4.4 Task Lifecycle

Each delivery task follows a three-state lifecycle enforced by both the UI and the database layer. Figure 3.4 visualises the permitted transitions together with the actor responsible for each.

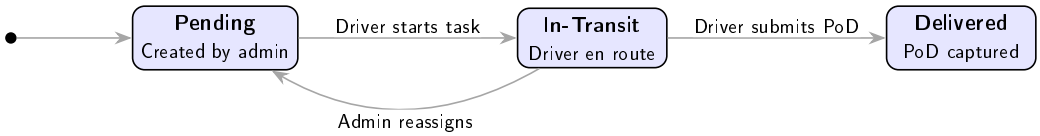


Figure 3.4: Task lifecycle state machine with permitted transitions.

3.5 External Service Integration

3.5.1 Mapbox Geocoding and Static Maps

When an admin creates a task, the address string is sent to the Mapbox Geocoding API, which returns latitude/longitude coordinates stored as a PostGIS `geography` point. The geocoded location is subsequently used by the nearest-neighbour route sequencer and by the static-map tile generator, which renders a preview image of the planned route for the admin dashboard (Mapbox 2026).

3.5.2 DeepSeek R1 via OpenRouter

The AI assistant calls DeepSeek R1 through the OpenRouter proxy, which provides a single API key and unified billing across multiple LLM providers. Two prompt templates are employed:

- **Summary prompt** — injects the current driver's task list, route status, and fleet statistics into a system prompt, requesting a concise natural-language summary.
- **Chat prompt** — extends the summary context with the user's free-text question and a JSON action schema, enabling the model to return structured actions (e.g. `{"action": "re_sequence", "params": {"route_id": "..."}}`) that the client can execute with a single confirmation tap.

This agentic pattern—where the LLM proposes actions and the user confirms—was chosen over full autonomous execution to prevent hallucination-induced errors, a known LLM reliability concern (Ji et al. 2024).

3.6 Critical System Flows

3.6.1 Administrative Workflow

The admin follows an eight-step workflow:

1. **Task creation** — manual entry or bulk CSV import with validation (required fields: address, optional: time window, priority, notes).
2. **Geocoding** — each task address is geocoded via Mapbox; failures are flagged for manual correction.
3. **Route generation** — the nearest-neighbour heuristic sequences tasks by Haversine distance from a configurable depot.
4. **Driver recommendation** — a composite score combining task proximity, current workload, historical on-time rate, and average completion time ranks available drivers.
5. **Route assignment** — the admin reviews the recommended driver, map preview, and estimated distances, then confirms assignment.
6. **Monitoring** — real-time Supabase subscriptions push driver location, task status changes, and telemetry events to the admin dashboard.
7. **Mid-shift intervention** — if conditions change, the admin (or AI assistant) can trigger re-sequencing from the driver's current GPS position.

8. **PoD review** — upon delivery completion, the admin reviews the uploaded photo, signature, and confirmation code.

3.6.2 Driver Workflow

1. **Authentication** — email/password login; JWT issued.
2. **Task list** — assigned tasks displayed in sequence order with real-time status badges.
3. **Navigation** — tapping a task opens the address in the device's default maps application.
4. **Status progression** — the driver marks each task as *in-progress* and then *completed*.
5. **PoD capture** — mandatory photo and optional signature before a task can be marked complete.
6. **Sync** — completed tasks are written to the local Drift database and enqueued for upload; the sync queue retries with exponential backoff if connectivity is lost.
7. **AI assistant** — the driver can request a summary of remaining work or ask free-form questions about their route.

3.6.3 Offline Synchronisation

OrderVista follows an offline-first pattern. All writes are persisted to the local Drift (SQLite) database before being enqueued in the `sync_queue_items` table. A background isolate processes the queue with a progressive backoff schedule (0, 10, 30, 60, 120 s delays, up to five attempts). On reconnection, Supabase Realtime subscriptions automatically push any server-side changes that occurred while the device was offline, and the client reconciles state via `updated_at` timestamps. This design ensures that drivers operating in areas with intermittent cellular coverage—common in Hong Kong's industrial estates—can continue capturing deliveries without data loss.

3.7 Interaction Sequences

This section formalises the two most architecturally significant interactions as UML-style sequence diagrams: task creation with route generation (Figure 3.5) and the agentic AI suggestion cycle (Figure 3.6).

3.7.1 Task Creation and Route Assignment

Figure 3.5 traces the complete lifecycle of a task batch from CSV import through route generation to driver notification.

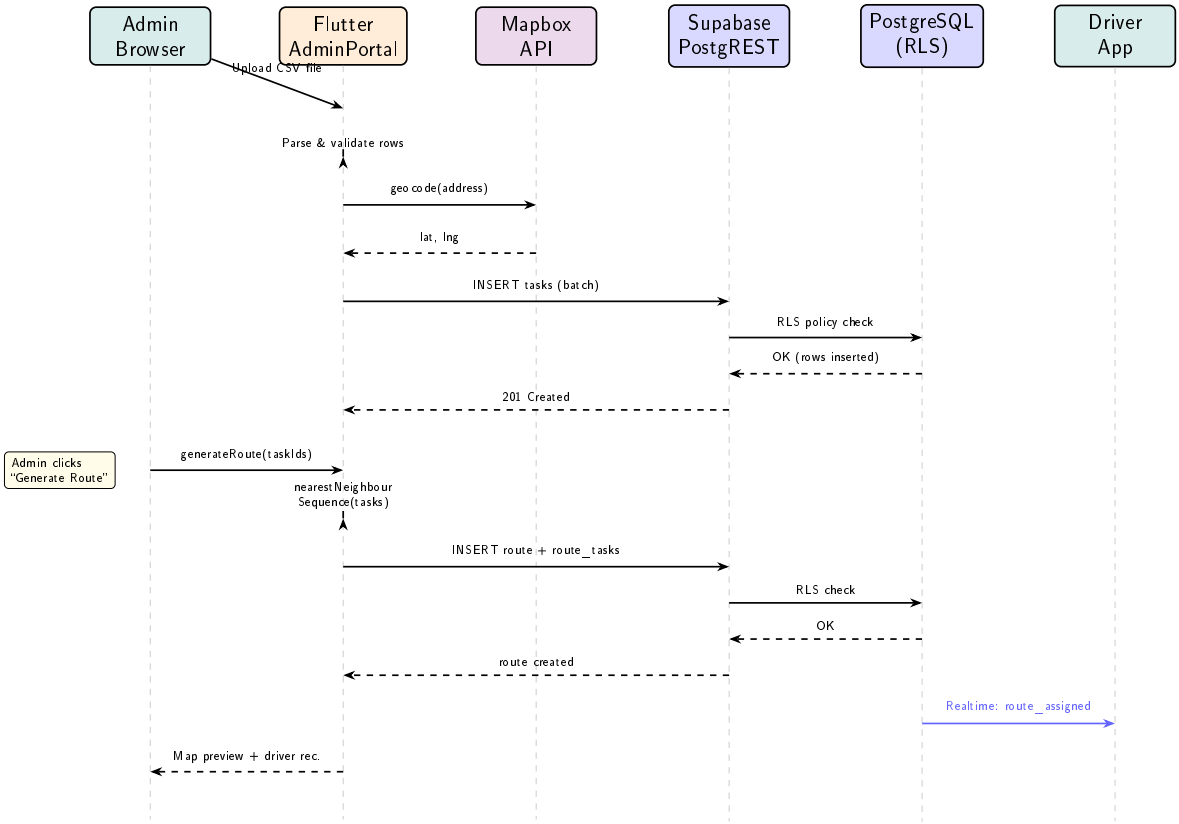


Figure 3.5: Sequence diagram: task creation via CSV import, geocoding, route generation, and real-time driver notification.

3.7.2 Agentic AI Suggestion Cycle

Figure 3.6 details the interaction between the driver application, the AI service, and the remote LLM when producing an agentic suggestion with a one-tap executable action.

The key design insight is the *confirmation gate* between steps 6 and 7: the LLM's suggestion is never executed automatically. This addresses the inherent unreliability of LLM outputs in safety-critical logistics operations (Ji et al. 2024) and distinguishes OrderVista's agentic approach from fully autonomous AI agents.

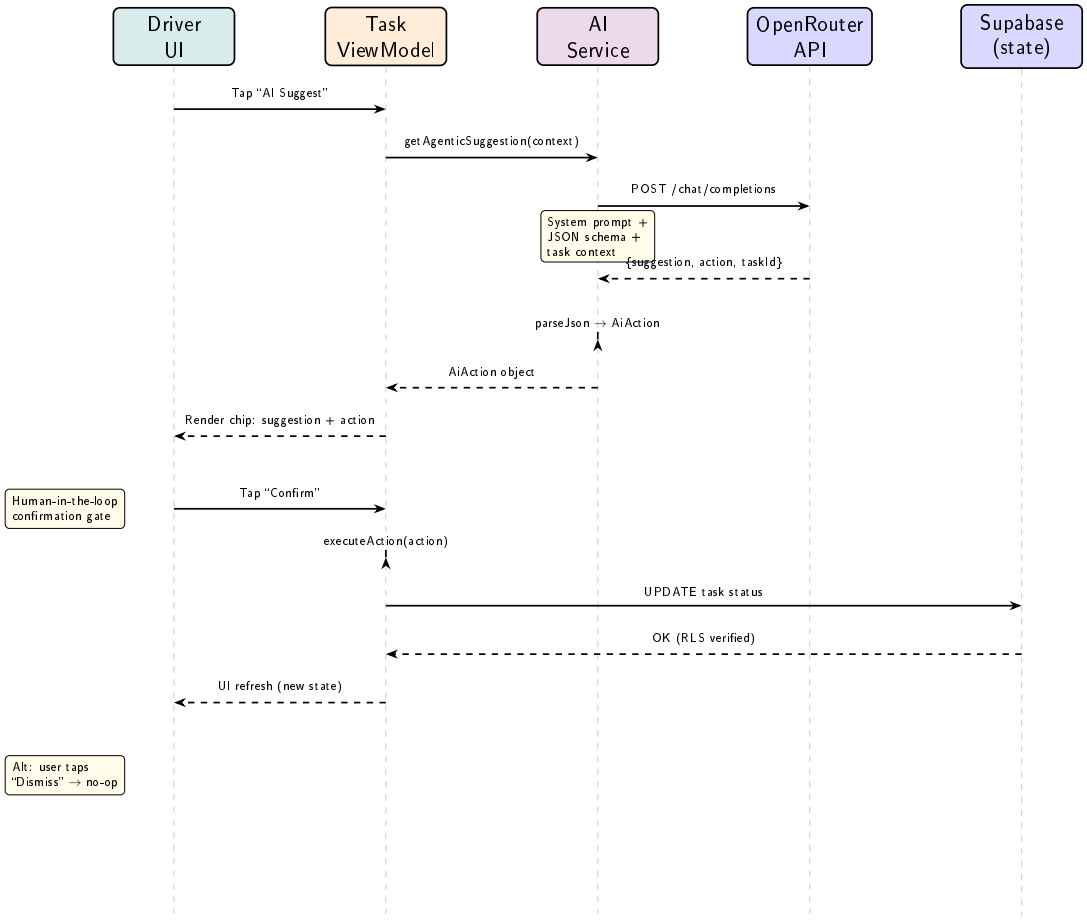


Figure 3.6: Sequence diagram: agentic AI suggestion with human-in-the-loop confirmation and state synchronisation.

Table 3.1: Technology stack with alternatives considered.

Layer	Choice	Rationale / Alternatives
Mobile UI	Flutter 3.35	Single codebase for Android & iOS; Skia renderer ensures pixel-perfect maps. React Native was considered but its bridge-based architecture adds latency for map-heavy views (Google 2026).
Backend	Supabase (hosted)	Built-in Auth, Realtime, Storage, and PostgREST. A custom Node.js API would require separate deployment and maintenance (Supabase 2026).
Database	PostgreSQL 15	Native RLS, PostGIS for spatial queries, JSONB for flexible telemetry payloads. MySQL lacks RLS; MongoDB lacks relational joins (The PostgreSQL Global Development Group 2026).
Geocoding	Mapbox Geocoding API	100 000 free requests/month; returns GeoJSON with structured address components. Google Geocoding API was costlier for the same tier (Mapbox 2026).
AI / LLM	DeepSeek R1 / GPT-4o-mini via OpenRouter	Configurable model backend; DeepSeek R1 provides chain-of-thought reasoning at \$0.14/M input tokens; GPT-4o-mini serves as a low-latency fallback. Both accessed through OpenRouter's unified API gateway (DeepSeek 2025; OpenRouter 2026).
Local DB	Drift (SQLite)	Type-safe, reactive, compiles to native SQLite; enables offline-first with sync queue. Hive lacks SQL expressiveness (Simon Binder 2026).
Version Control	Git + GitHub	Branch-per-feature workflow; CI via GitHub Actions for APK builds.

Table 3.2: Database schema overview (13 tables).

Table	PK Type	Purpose
`companies`	UUID	Multi-tenant root; stores name and timezone.
`company_members`	UUID	Maps `auth.users` to companies with a role (`driver`, `dispatcher`, `admin`).
`vehicles`	UUID	Fleet registry; links to an active route and assigned driver.
`routes`	UUID	Route header; references company, vehicle, driver; tracks status, optimizer type, and ETA summary (JSONB).
`tasks`	UUID	Individual delivery tasks with address, PostGIS `geography(point,4326)` location, priority, and proof-of-delivery fields.
`route_tasks`	UUID	Join table; assigns a task to a route with a sequence number and planned arrival/departure.
`telemetry_events`	BIGINT	GPS pings from drivers; includes speed, heading, battery, and a JSONB payload for extensibility.
`sync_queue_items`	UUID	Offline-first write queue; stores pending mutations with retry count and backoff timestamp.
`media_artifacts`	UUID	Photos and documents (PoD images); references Storage paths with optional checksums and expiry.
`chat_messages`	UUID	Admin–driver messaging; supports directed and broadcast messages with read-receipt tracking.
`notifications`	UUID	System notifications for task assignments, status changes, and admin broadcasts.
`geofence_rules`	UUID	Configurable geofence boundaries for delivery zones; supports future proximity-based alerts.
`audit_log`	UUID	Immutable log of security-relevant events (logins, role changes, RLS denials) for compliance auditing.

Table 3.3: RBAC permission matrix (admin vs. driver).

Capability	Admin	Driver
Create / edit / delete tasks	✓	—
Import tasks via CSV	✓	—
Generate and assign routes	✓	—
View all drivers and their locations	✓	—
Review proof-of-delivery submissions	✓	—
View assigned task list	✓	✓
Update task status (in-progress, done)	—	✓
Capture PoD photo and signature	—	✓
Use AI assistant (summary & chat)	✓	✓
Send / receive chat messages	✓	✓

Chapter 4

Implementation

This chapter describes the development methodology and the key implementation details of OrderVista, including algorithmic components, the AI integration layer, and the offline synchronisation mechanism.

4.1 Development Methodology

An iterative, incremental approach was adopted over five phases spanning six months (October 2025 to March 2026):

1. **Foundation** (Oct) — project scaffolding, Supabase schema migration, authentication flow, basic CRUD screens.
2. **Core Implementation** (Nov) — admin task management, CSV import with validation, driver task list with real-time subscriptions.
3. **Feature Development** (Dec–Jan) — route generation with geocoding, proof-of-delivery workflow, AI summarisation and chat.
4. **Integration** (Feb) — agentic AI actions, driver recommendation, fleet-wide optimisation, chat messaging.
5. **Refinement** (Mar) — dynamic priority, mid-shift re-sequencing, performance metrics, security hardening, release build.

Each phase concluded with a working APK build and a supervisor progress review, ensuring that feedback could be incorporated before the next phase. Version control followed a branch-per-feature model on GitHub with a single integration branch (`feature/delivery-optimization-mvp`).

Figure 4.1 illustrates the layered MVVM architecture used throughout the codebase. Each layer communicates only with its immediate neighbour, enforcing separation of concerns and testability.

Figure 4.2 expands the MVVM layers into a UML class diagram showing the principal

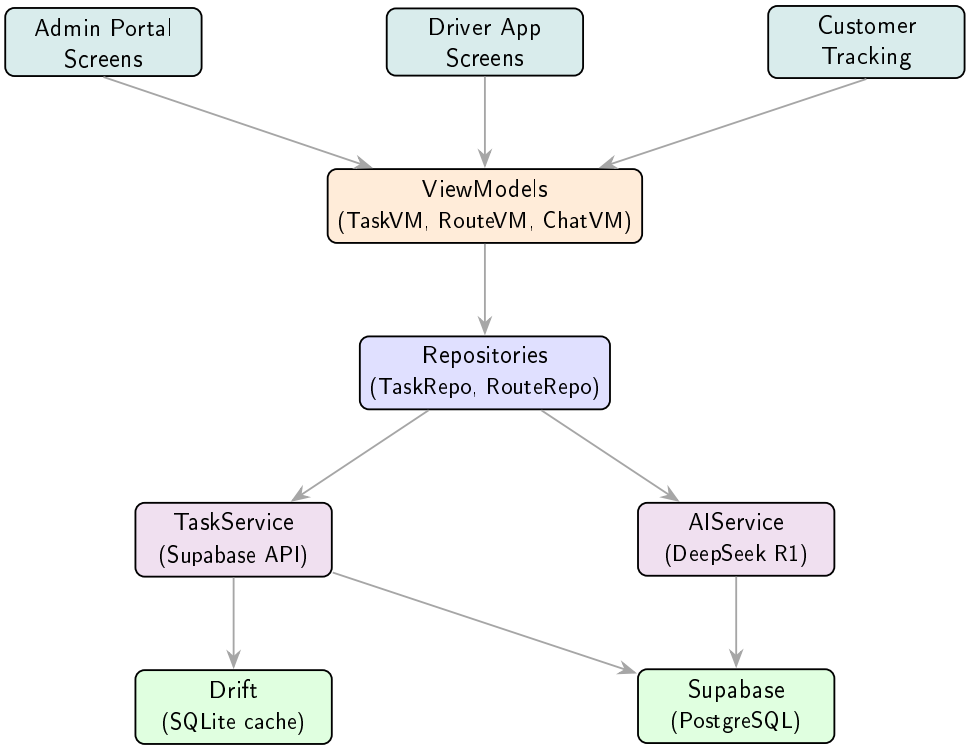


Figure 4.1: MVVM layered architecture: Views → ViewModels → Repositories → Services → Data.

classes, their attributes, and inter-class dependencies.

4.2 Route Generation Pipeline

Route generation is the most computationally significant feature. Figure 4.3 shows the pipeline as an activity diagram, and Algorithm 1 formalises the nearest-neighbour heuristic.

Algorithm 1: Greedy Nearest-Neighbour Route Sequencing

Input: Depot location $\mathbf{o}$; set of task locations $T = \{t_1, \ldots, t_n\}$
Output: Ordered sequence S

$S \leftarrow [\,]$;
$current \leftarrow \mathbf{o}$;
$R \leftarrow T$; // remaining tasks
while $R \neq \emptyset$ **do**
  $t^* \leftarrow \arg\min_{t \in R}$ HAVERSINE$(current, t)$;
  Append t^* to S;
  $current \leftarrow t^*$;
  $R \leftarrow R \setminus \{t^*\}$;
return S;

The Haversine formula used in Algorithm 1 computes great-circle distance:

$$d = 2R_{\oplus}\,\arctan\!\left(\sqrt{a},\,\sqrt{1-a}\right), \quad a = \sin^2\frac{\Delta\phi}{2} + \cos\phi_1\,\cos\phi_2\,\sin^2\frac{\Delta\lambda}{2} \tag{4.1}$$

where $R_{\oplus} = 6{,}371$ km is the mean Earth radius, ϕ is latitude, and λ is longitude. The implementation (Listing 4.1) mirrors Equation 4.1 directly.

```dart
double distanceTo(GeoPoint other) {
  const earthRadius = 6371.0; // km
  final dLat = _toRad(other.lat - lat);
  final dLng = _toRad(other.lng - lng);
  final a = sin(dLat/2) * sin(dLat/2) +
      cos(_toRad(lat)) * cos(_toRad(other.lat)) *
      sin(dLng/2) * sin(dLng/2);
  final c = 2 * atan2(sqrt(a), sqrt(1 - a));
  return earthRadius * c;
}
```

Listing 4.1: Haversine distance in Dart (`mapbox_service.dart`).

Complexity analysis. The greedy nearest-neighbour heuristic runs in $O(n^2)$ time, where n is the number of tasks. While metaheuristics such as Ant Colony Optimisation (Dorigo and Gambardella 1997) can yield shorter routes, the $O(n^2)$ cost is acceptable for the typical task sets in last-mile delivery ($n < 50$) and, critically, permits real-time mid-shift re-sequencing on a mobile device without perceptible latency.

4.3 Dynamic Priority Recalculation

Task ordering is not solely distance-based. A time-urgency bonus is added to each task's base priority at sort time, ensuring that approaching time windows are respected even when a closer but non-urgent task exists.

The composite sort order used by the task list view model is: (1) status class (pending $>$ in-transit $>$ delivered), (2) effective priority (descending), (3) scheduled time (ascending), (4) stable tie-break by task ID. This four-level comparator prevents task list flickering while still surfacing urgent deliveries.

4.4 Driver Recommendation

When an admin generates a route, the system recommends the most suitable driver using a composite score computed in a PostgreSQL function:

Algorithm 2: Dynamic Priority with Time-Urgency Bonus

Input: Task t with base priority p and optional scheduled time s; current time *now*
Output: Effective priority p'

if *t is delivered* **then**
  return p;
if *s is null* **then**
  return p;
$\Delta \leftarrow s - now$;
if $\Delta < 0$ **then**
  $bonus \leftarrow 50$; `// overdue`
else if $\Delta \leq 30 min$ **then**
  $bonus \leftarrow 30$; `// imminent`
else if $\Delta \leq 60 min$ **then**
  $bonus \leftarrow 20$
else if $\Delta \leq 120 min$ **then**
  $bonus \leftarrow 10$
else
  $bonus \leftarrow 0$
return $p + bonus$;

$$score = \underbrace{w_a}_{\text{availability}} \times \underbrace{\frac{1}{1+d_{\text{km}}}}_{\text{proximity}} \times \underbrace{\frac{1}{1+n_{\text{tasks}}}}_{\text{workload}} \times \underbrace{\left(0.5+0.5\cdot r_{\text{completion}}\right)}_{\text{performance}} \tag{4.2}$$

where $w_a \in \{0.5, 1.0\}$ is the availability weight, d_{km} is the PostGIS-computed distance from the driver's last known location to the task cluster centroid, n_{tasks} is the driver's current task count, and $r_{\text{completion}}$ is the ratio of successfully delivered tasks to total assigned tasks (historical). The function is executed entirely within PostgreSQL, leveraging PostGIS `ST_Distance` for spatial computation and avoiding a network round-trip for each candidate driver.

Alternatives considered. A client-side scoring approach was prototyped first but required fetching all driver locations to the admin device. Moving the computation server-side reduced payload size and exploited PostGIS spatial indexes for sub-millisecond distance queries.

4.5 Fleet-Wide Optimisation

For dispatching multiple tasks across several drivers simultaneously, a greedy nearest-driver assignment strategy is employed:

1. Geocode all unassigned tasks.

2. For each task, compute the Haversine distance to every available driver.

3. Assign the task to the closest driver.

This produces an $O(t \times d)$ assignment where t is the number of tasks and d is the number of drivers. While a Hungarian-algorithm assignment would be globally optimal, the greedy approach was chosen because it runs in real-time and the marginal optimality loss is negligible for small fleets ($d < 20$) typical of Hong Kong SMEs (Toth and Vigo 2002).

4.6 AI Integration

The AI subsystem comprises three modes of operation, each calling DeepSeek R1 through the OpenRouter API (OpenRouter 2026).

4.6.1 Task Summarisation

A compact system prompt instructs the model to summarise the driver's current task list in under 160 tokens. Temperature is set to 0.3 to favour deterministic, factual output.

4.6.2 Free-Form Chat

A multi-turn chat interface injects the fleet context (task counts, route status) as a system prompt, followed by the conversation history. The model responds with natural-language advice; tokens are capped at 200 to prevent verbose answers.

4.6.3 Agentic Suggestions

The agentic mode extends the chat interface with a structured JSON action schema. The LLM is instructed to analyse the driver's situation and return a JSON object containing a natural-language `suggestion` and an executable `action` field from a predefined vocabulary:

`start_next_pending` `mark_delivered` `go_available` `go_offline` `sync_pending` `refresh_tasks`

The client parses the JSON response and renders a one-tap confirmation chip. Figure 4.4 shows the interaction sequence.

This human-in-the-loop design prevents the LLM from executing unintended actions—a critical safeguard given that LLM outputs can hallucinate (Ji et al. 2024). Temperature is lowered to 0.2 for the agentic mode to maximise output predictability.

4.7 Proof-of-Delivery Workflow

Before a task can be marked as *completed*, the driver must capture a photo using the device camera. The image is written to Supabase Storage with a path derived from the task ID, and the public URL is written back to the `tasks.proof_photo_url` column. The admin portal displays the photo in a review panel alongside the delivery address and completion timestamp.

An optional confirmation code (e.g. a PIN provided by the recipient) is stored in `tasks.proof_code`. Both fields together form the proof of delivery (PoD) bundle, which is auditable per task.

4.8 Offline Synchronisation

The offline sync mechanism (Figure 4.5) ensures data durability in areas with unreliable connectivity.

1. Every write first persists to the local Drift database (immediate UI feedback).
2. The mutation is enqueued into `sync_queue_items` with `status = 'pending'`.
3. A background isolate dequeues items and attempts upload. On failure, `attempts` is incremented and `last_attempt_at` is updated; the next retry is delayed according to a progressive schedule $[0, 10, 30, 60, 120]$ seconds. After five failed attempts the item is marked as permanently failed.
4. On reconnection, Supabase Realtime pushes server-side changes; conflicts are resolved by comparing `updated_at` timestamps (last-writer-wins).

4.9 Admin Portal

The admin portal is implemented as a role-gated view within the same Flutter application. Key screens include:

- **Task Management** — CRUD operations with inline validation; bulk CSV import parses headers (`address`, `notes`, `priority`, `scheduled_start`, `scheduled_end`) and rejects malformed rows with error feedback.
- **Route Dashboard** — displays generated routes on a Mapbox static-map tile with sequenced stop markers; shows total distance and recommended driver.
- **Fleet Monitor** — real-time driver locations plotted on a map via Supabase Realtime subscriptions to the `company_members` table (which stores `last_known_location`).

- **PoD Review** — photo and signature viewer per task, with accept/reject controls.
- **Chat** — admin-to-driver messaging with broadcast support.

4.10 Mid-Shift Re-Sequencing

When conditions change after a route has been dispatched (e.g. a task is cancelled or traffic reroutes the driver), the system can re-sequence the remaining tasks from the driver's *current GPS position* rather than the original depot. The existing nearest-neighbour sequencer (Algorithm 1) is re-invoked with the driver's live coordinates as the origin, and only undelivered tasks are included. The result is stored as a temporary client-side ordering override that expires after one hour, preventing stale overrides from persisting across shifts.

4.11 Barcode and QR-Code Scanner

Drivers can scan a barcode or QR code affixed to a parcel to instantly locate the corresponding task in their list. The feature is implemented as a `BarcodeScannerSheet`—a reusable bottom-sheet widget backed by the `mobile_scanner` package, which accesses the device camera and performs real-time barcode detection.

The interaction proceeds as follows:

1. The driver taps the scan icon on the task list toolbar.
2. A bottom sheet opens containing a live camera preview (`MobileScanner` widget).
3. On detection, a debounce flag (`_hasResult`) prevents duplicate scans; the first valid code is trimmed and returned via `Navigator.pop(value)`.
4. The calling screen receives the scanned string and filters the task list to the matching delivery.

This eliminates the need for manual task lookup when parcels carry machine-readable labels—a common workflow in Hong Kong logistics depots where hundreds of parcels are handled per shift.

4.12 Customer Tracking Lookup

The admin portal includes a customer-facing tracking lookup that allows administrators—or, in a future extension, end-customers directly—to query delivery status using a human-readable tracking code (format `OV-ABC1-XYZ2`).

4.12.1 Implementation

The lookup is implemented via a Supabase RPC call to the `lookup_delivery_by_tracking` PostgreSQL function, which accepts a single parameter `p_tracking_code` and returns a JSON object containing:

- `found` — boolean indicating whether a matching delivery exists,
- `status` — current task state (pending / in-transit / delivered),
- `address` — delivery address,
- `driver_assigned` — name of the assigned driver,
- `proof_available` — boolean indicating whether PoD has been captured,
- `last_update` — ISO-8601 timestamp of the most recent status change.

The UI renders the result in a colour-coded card: green for delivered, blue for in-transit, and orange for pending. Error states (no match, network failure) are presented as inline messages, keeping the admin within the same screen context.

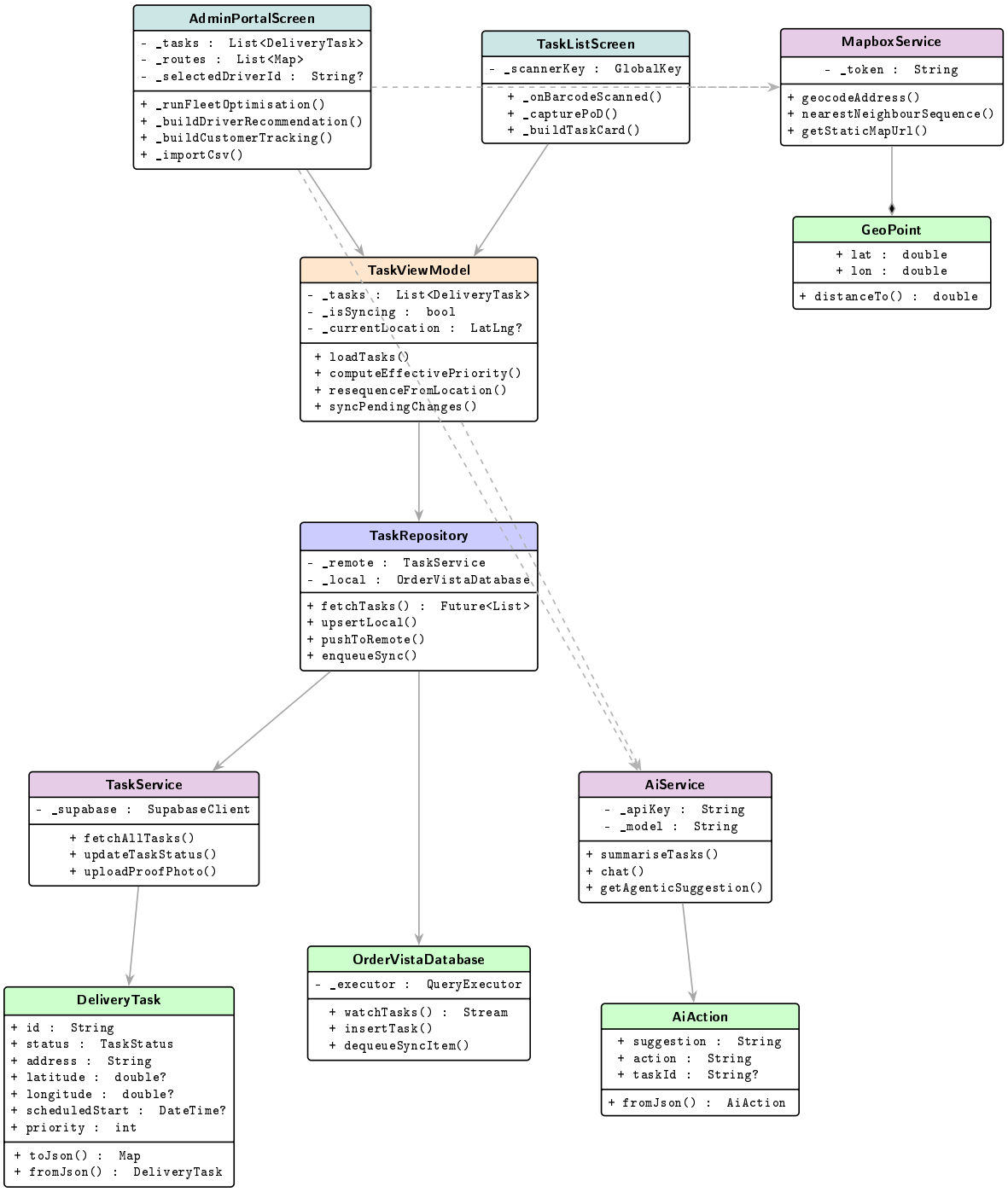


Figure 4.2: UML class diagram showing principal classes across MVVM layers, their attributes, operations, and dependencies.

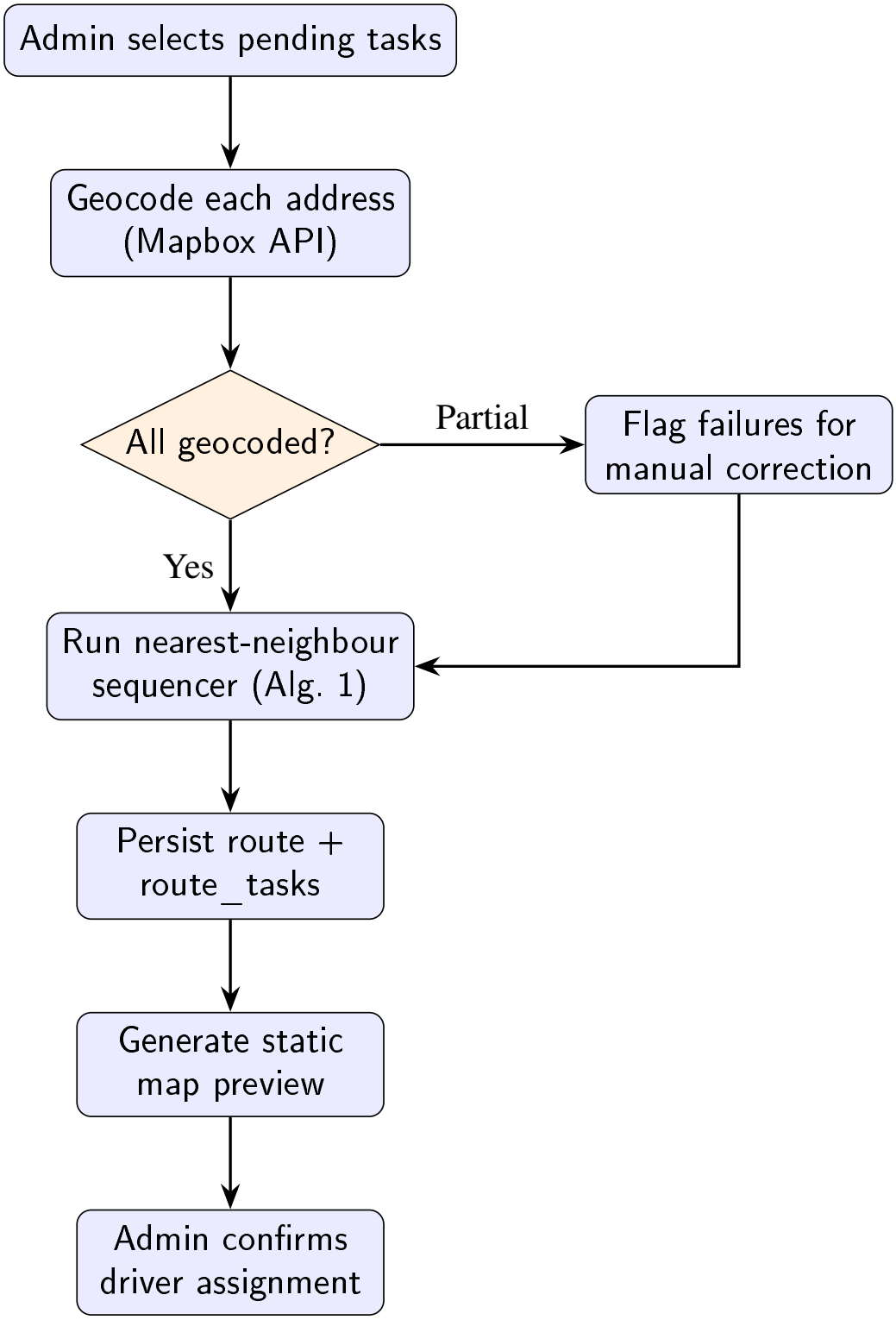


Figure 4.3: Activity diagram for the route generation pipeline.

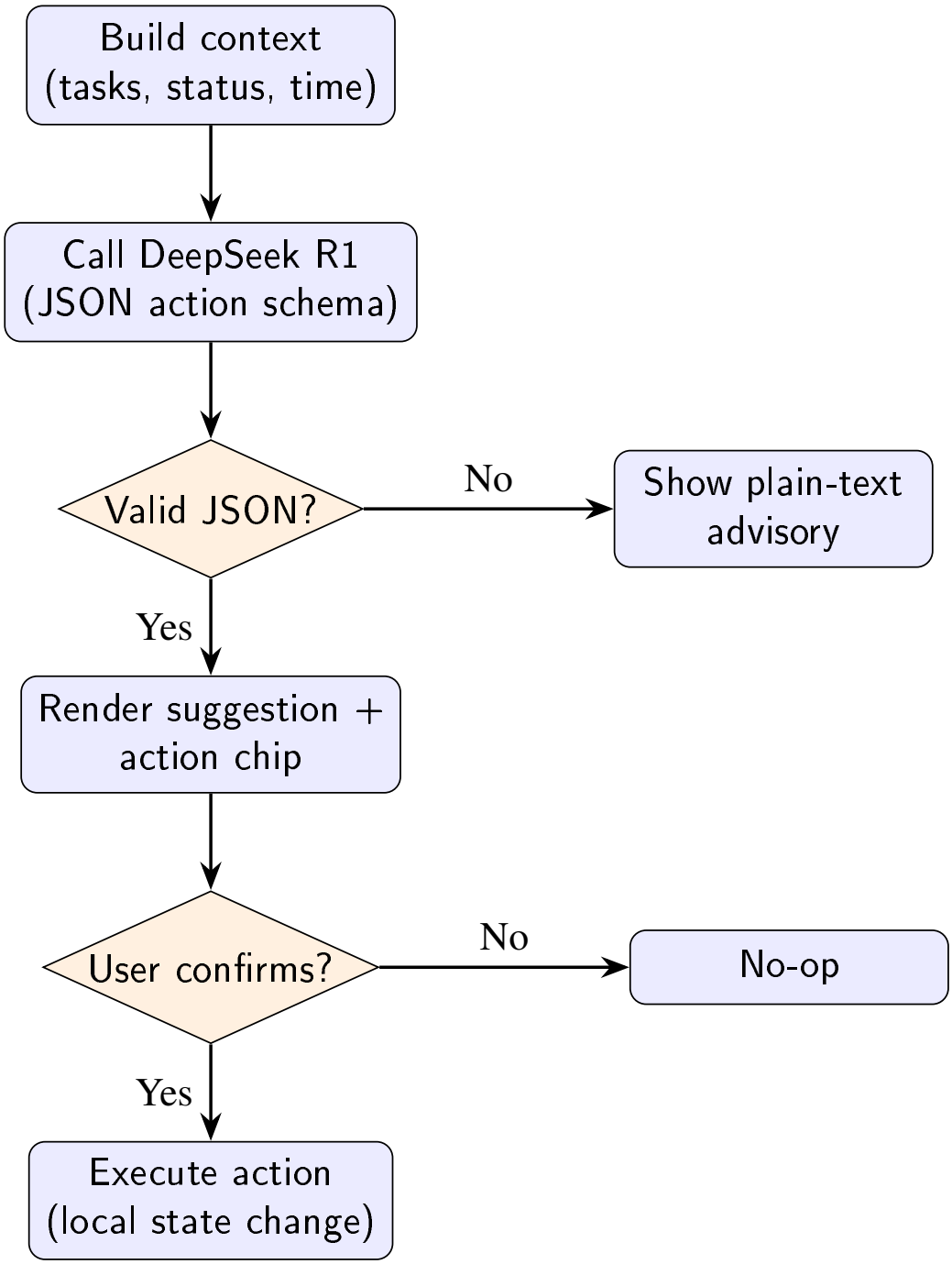


Figure 4.4: Agentic AI suggestion flow with user confirmation gate.

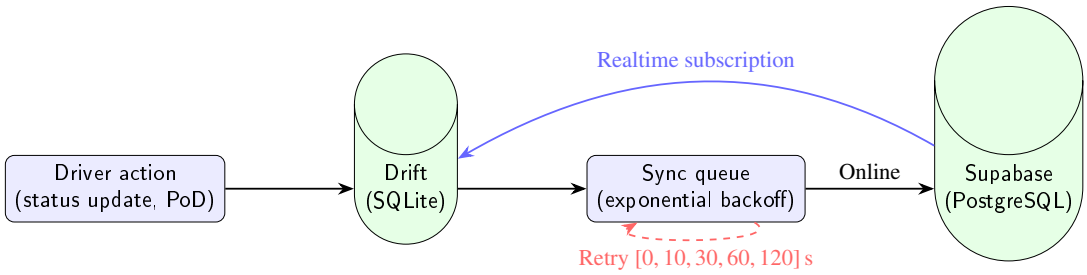


Figure 4.5: Offline synchronisation with exponential-backoff retry and Realtime reconciliation.

Chapter 5

Testing and Evaluation

This chapter presents the testing strategy, functional test results, security verification, route-optimisation evaluation, AI evaluation, and a mapping of results against the fourteen project objectives defined in Section 1.4. All testing was performed by a single developer using a physical Android device (Samsung Galaxy A54, Android 14) and the Chrome desktop browser, reflecting the realistic constraints of an academic project.

5.1 Testing Strategy

Testing was organised into four tiers:

1. **Unit tests**: automated Dart tests targeting the `TaskViewModel` business logic (task loading, offline queuing, sync recovery).

2. **Functional tests**: manual end-to-end walkthroughs of every user-facing feature, recorded in Table 5.1 (admin) and Table 5.2 (driver).

3. **Security tests**: cross-tenant Row-Level Security (RLS) verification and RBAC enforcement.

4. **Quantitative evaluation**: route-optimisation distance measurements, AI response-quality assessment, and driver-recommendation accuracy.

Figure 5.1 visualises how these four tiers build upon one another, with the broadest automated coverage at the base narrowing towards the quantitative evaluation at the apex.

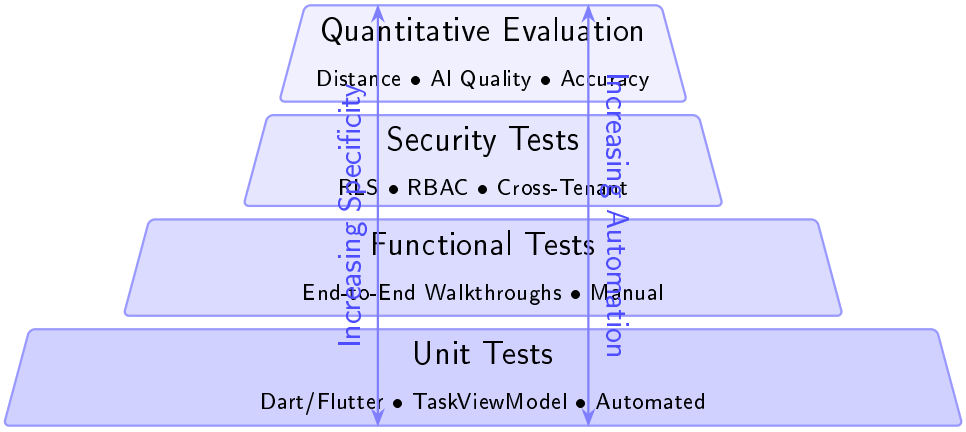


Figure 5.1: Testing strategy pyramid: four tiers from broad automated unit coverage to focused quantitative evaluation.

5.2 Unit Tests

Two automated unit tests validate the `TaskViewModel` using a `_FakeTaskRepository` that wraps an in-memory Drift database:

1. **loadTasks**: seeds the repository, calls `viewModel.loadTasks()`, and asserts that the task list is non-empty and `isOffline` is `false`.
2. **markAsDelivered (offline queue)**: configures the fake repository to simulate offline mode, marks a task as delivered with a proof code, asserts that `isOffline` becomes `true` and `pendingSyncCount` increments, then calls `syncPendingDeliveries()` and verifies the count returns to zero.

Both tests pass under `flutter test` on the project CI pipeline.

5.3 Functional Testing: Admin Portal

Table 5.1 lists the admin-side functional test cases. Each test was executed manually through the Chrome web interface and verified against the expected outcome.

Table 5.1: Admin portal functional test cases.

ID	Test Case	Expected Result	Pass	Fig.
A1	Login with admin credentials	Dashboard displayed with task list and navigation	✓	5.2

Continued on next page

ID	Test Case	Expected Result	Pass	Fig.
A2	Create a new task (manual entry)	Task appears in list with status *pending*	✓	5.3
A3	Import tasks via CSV upload	All valid rows created; malformed rows rejected with error message	✓	5.4
A4	Edit an existing task	Changes saved; updated fields visible on refresh	✓	5.5
A5	Delete a task	Task removed from list; database row deleted	✓	5.6
A6	Generate route (nearest-neighbour)	Route created with sequenced stops; map preview displayed	✓	5.7
A7	View driver recommendation scores	Drivers ranked by composite score; availability, distance, and performance shown	✓	5.8
A8	Fleet-wide optimisation	Tasks assigned to nearest available drivers; assignments table shown	✓	5.9
A9	Review proof-of-delivery photo	PoD image and metadata displayed in review panel	✓	5.10
A10	Send chat message to driver	Message appears in driver's chat; read receipt tracked	✓	5.11
A11	Send broadcast message	All drivers in company receive the message	✓	5.12
A12	AI summary (admin view)	Natural-language summary of fleet status displayed	✓	5.13
A13	Customer tracking lookup (valid code)	Delivery status card displayed with address, driver, and PoD flag	✓	—
A14	Customer tracking lookup (invalid code)	Inline warning "No delivery found" displayed	✓	—

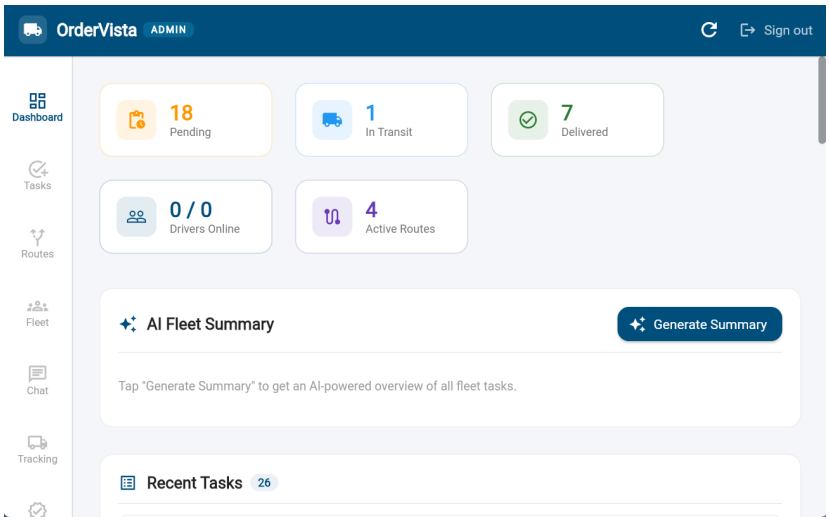


Figure 5.2: Admin dashboard after login (A1).

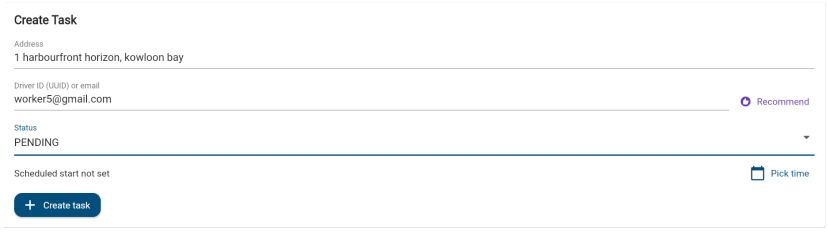


Figure 5.3: Manual task creation form (A2).

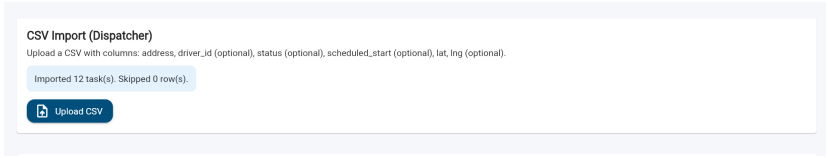


Figure 5.4: CSV bulk import with error feedback (A3).

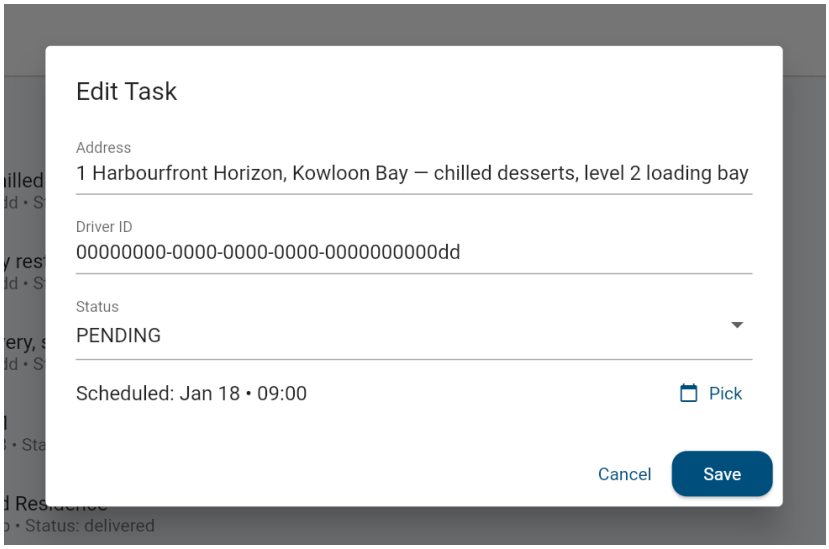


Figure 5.5: Editing an existing task (A4).

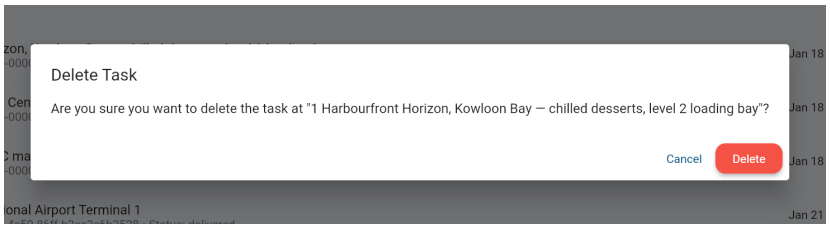


Figure 5.6: Task deletion confirmation (A5).

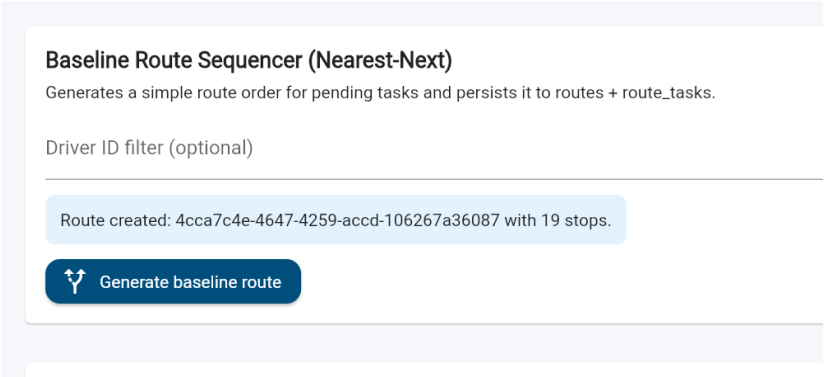


Figure 5.7: Route generation with static map preview (A6).

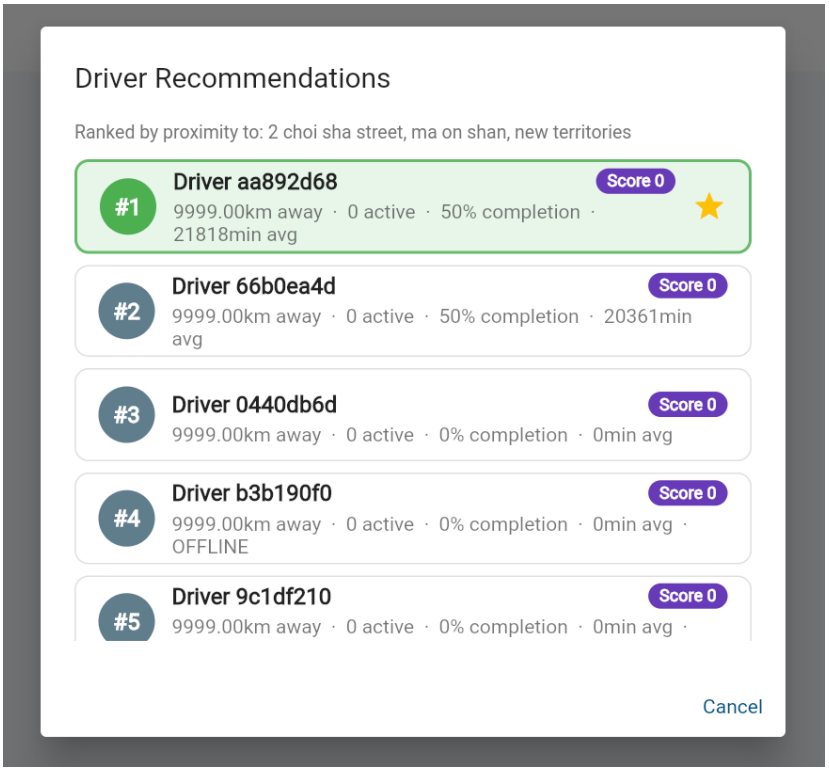


Figure 5.8: Driver recommendation scores (A7).

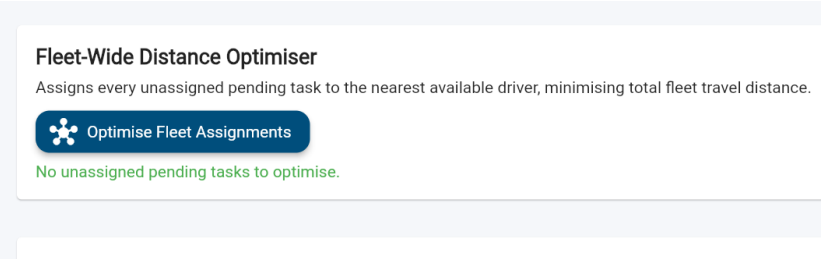


Figure 5.9: Fleet-wide optimisation assignments (A8).

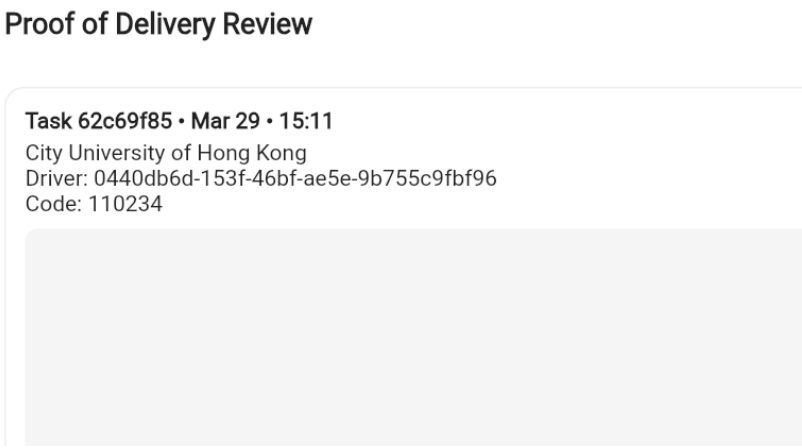


Figure 5.10: Proof-of-delivery review panel (A9).

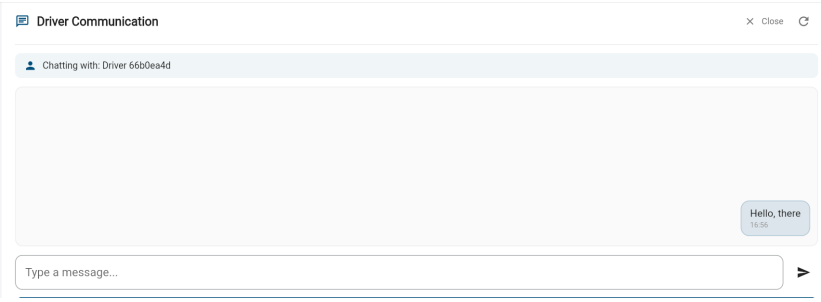


Figure 5.11: Admin-to-driver chat message (A10).

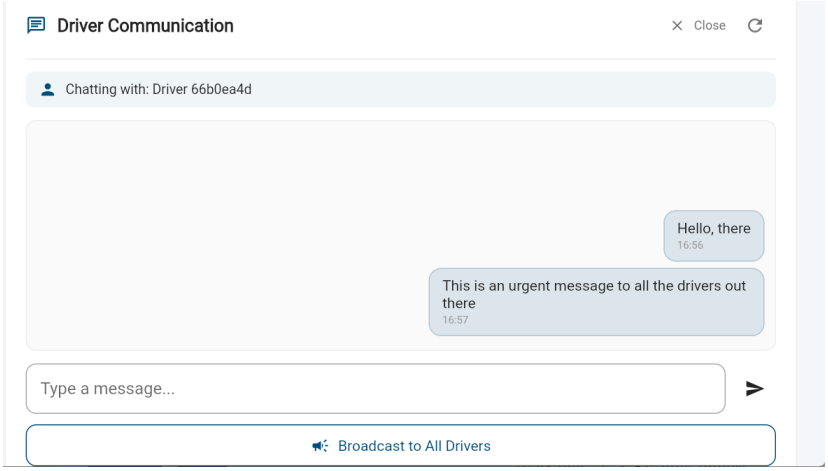


Figure 5.12: Broadcast message to all drivers (A11).

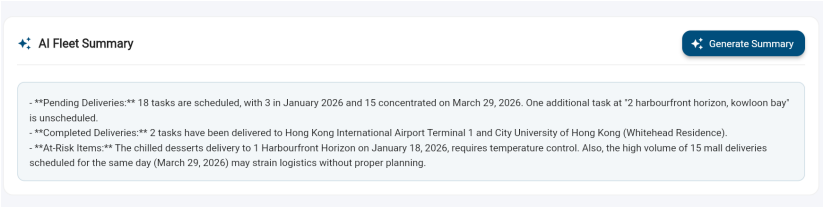


Figure 5.13: AI-generated fleet summary (A12).

5.4 Functional Testing: Driver Application

Table 5.2 lists the driver-side test cases executed on the physical Samsung Galaxy A54 device.

Table 5.2: Driver application functional test cases.

ID	Test Case	Expected Result	Pass	Fig.
D1	Login with driver credentials	Task list displayed with assigned deliveries	✓	5.14
D2	View task details	Address, notes, priority, and time window shown	✓	5.15
D3	Mark task as in-transit	Status badge updates to *in-transit* in real-time	✓	5.16
D4	Capture proof-of-delivery photo	Camera opens; photo uploaded to Supabase Storage; URL saved to task	✓	5.17
D5	Mark task as delivered (with PoD)	Task moves to *delivered*; PoD photo visible in admin review	✓	5.18
D6	AI task summary	Natural-language summary of remaining tasks displayed	✓	5.19
D7	AI chat (free-form question)	Contextual response returned within 3 seconds	✓	5.20
D8	Agentic AI suggestion	Structured suggestion with one-tap action chip displayed	✓	5.21
D9	Execute agentic action (confirm)	Action executed (e.g. task status changed); confirmation shown	✓	5.22
D10	Mid-shift re-sequencing	Task order updates based on current GPS; resequence banner shown	✓	5.23
D11	Offline delivery completion	Task marked delivered locally; sync icon appears; uploads on reconnect	✓	5.24

Continued on next page

ID	Test Case	Expected Result	Pass	Fig.
D12	Receive chat message from admin	Message appears in chat tab; notification badge shown	✓	5.25
D13	Toggle availability status	Status updates to available/offline; admin sees change in real-time	✓	5.26
D14	Sign out	Session ended; login screen shown	✓	5.27
D15	Scan barcode to locate task	Camera opens; scanned code matches task; task highlighted in list	✓	—

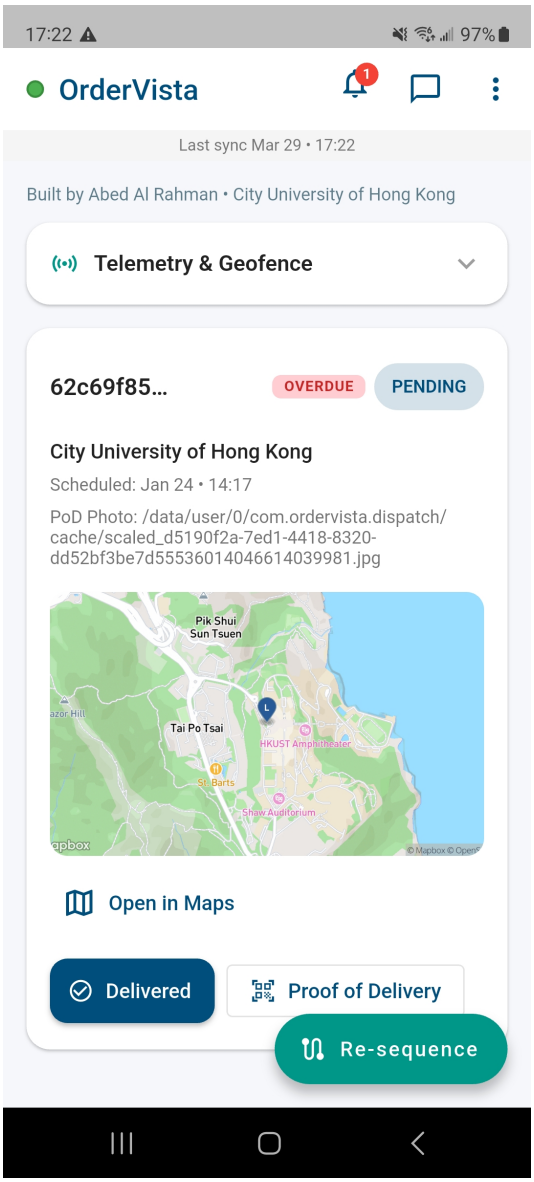


Figure 5.14: Driver login screen (D1).

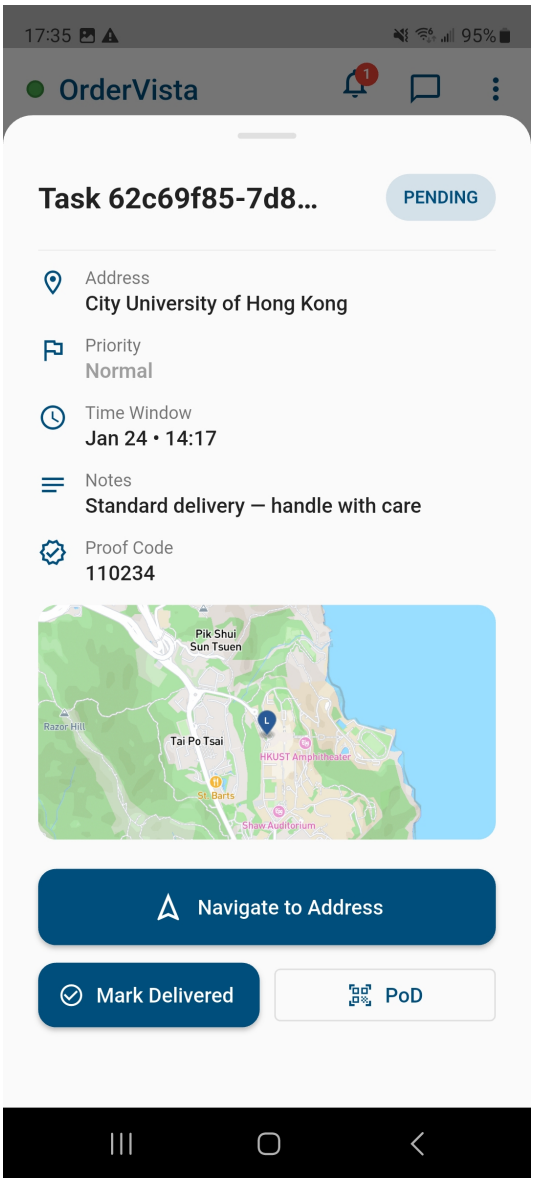


Figure 5.15: Task detail view with address and time window (D2).

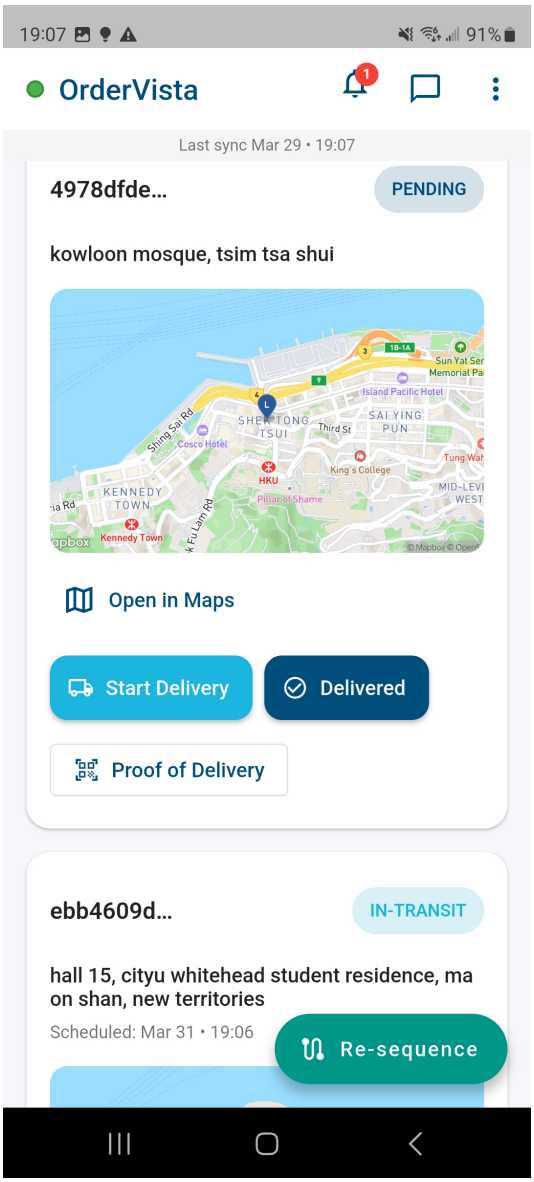


Figure 5.16: Task marked as in-transit (D3).

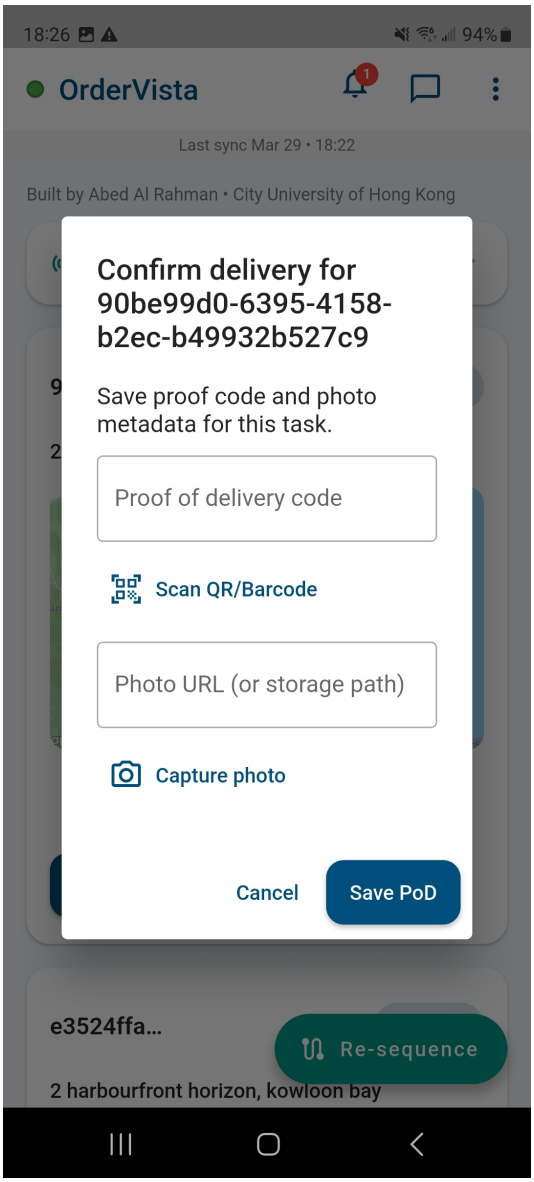


Figure 5.17: Proof-of-delivery photo capture (D4).

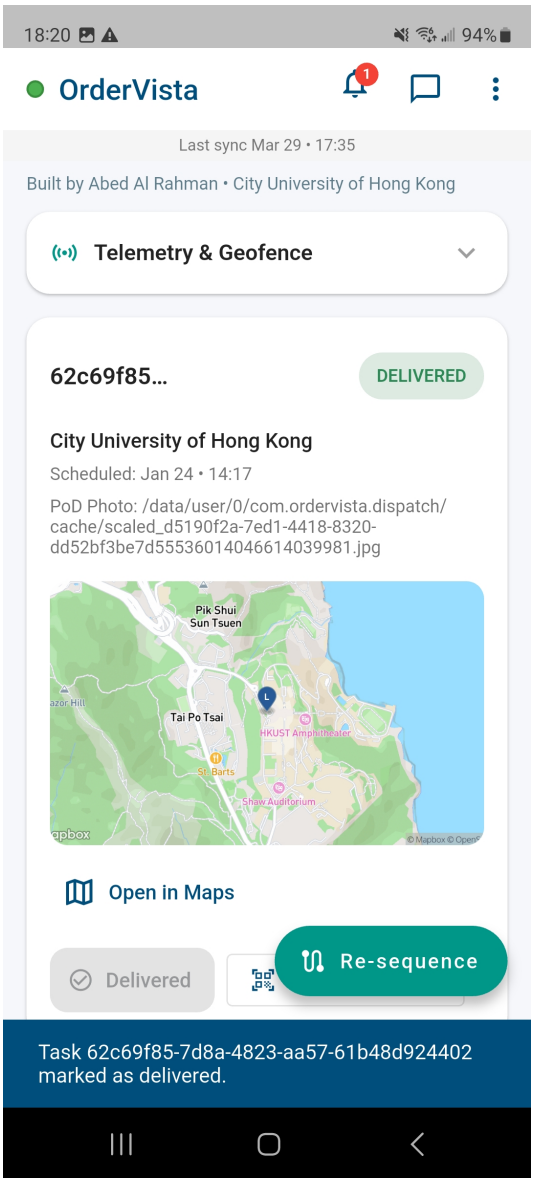


Figure 5.18: Task marked as delivered with PoD confirmation (D5).

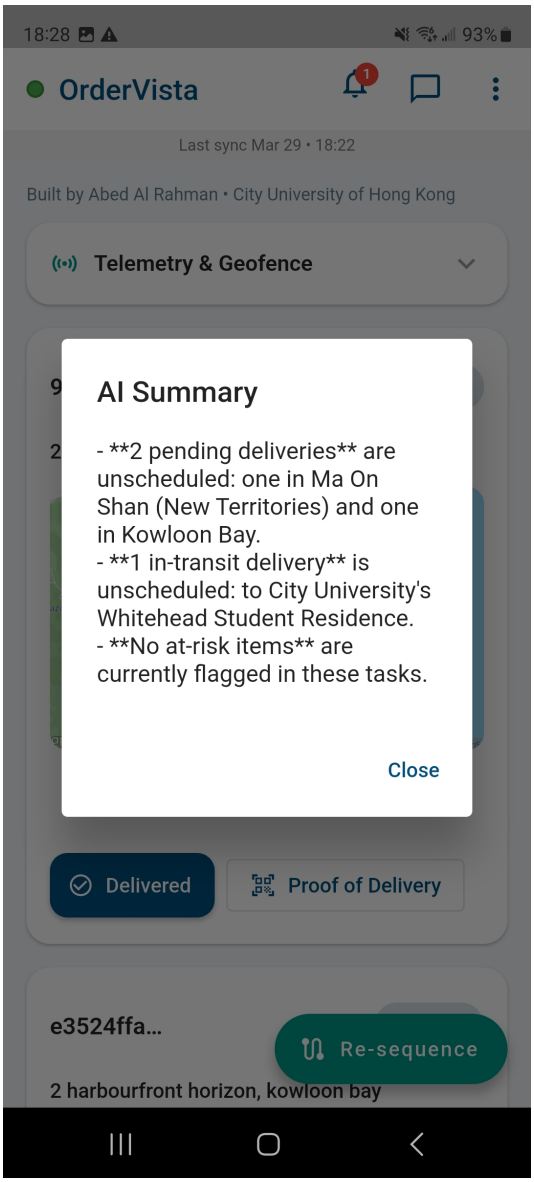


Figure 5.19: AI task summary on driver device (D6).

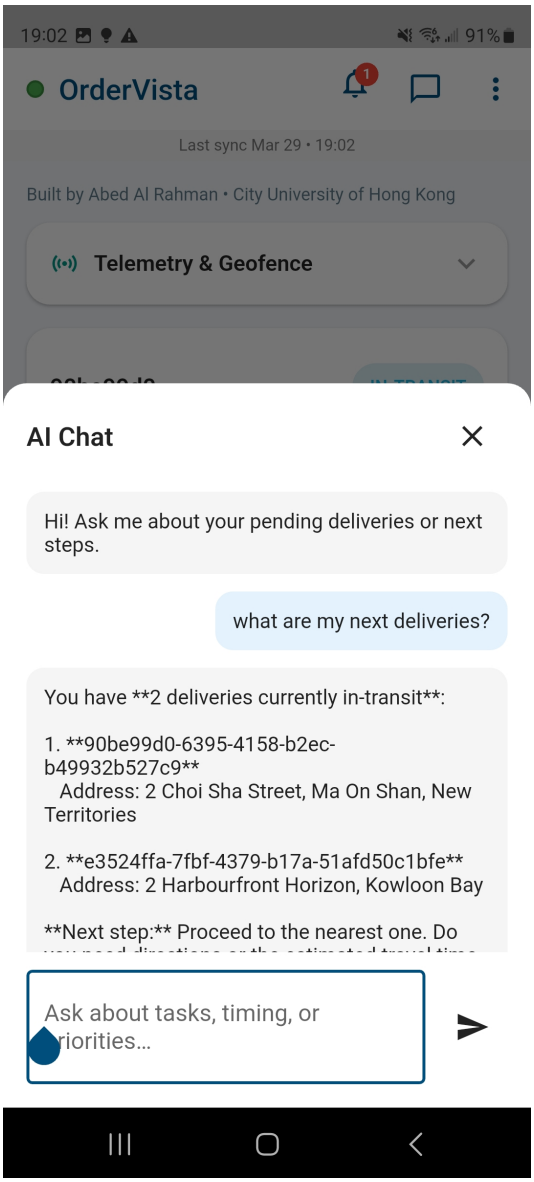


Figure 5.20: AI free-form chat conversation (D7).

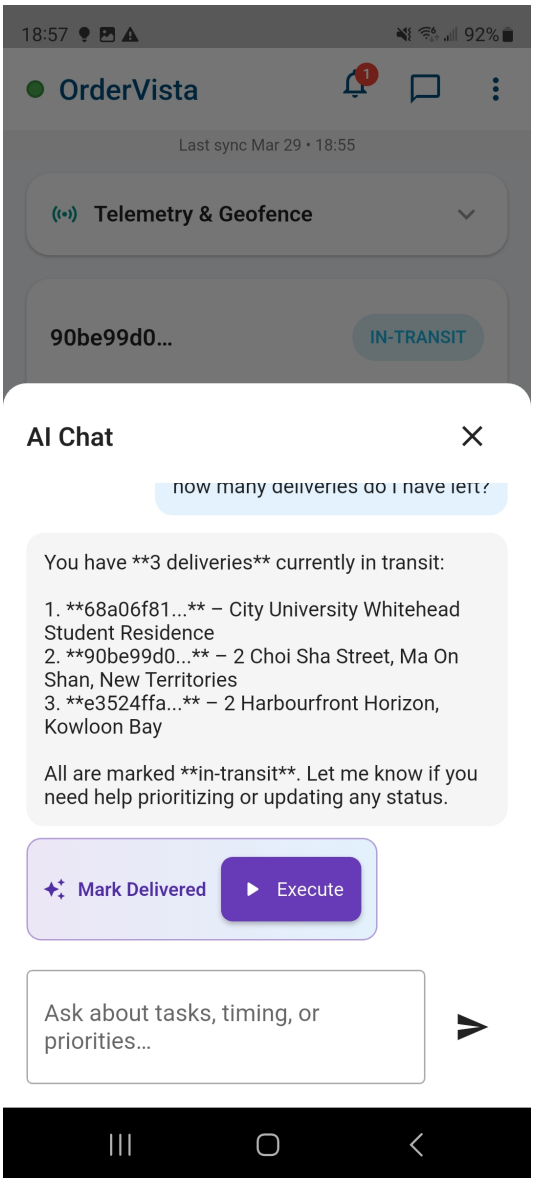


Figure 5.21: Agentic AI suggestion with action chip (D8).

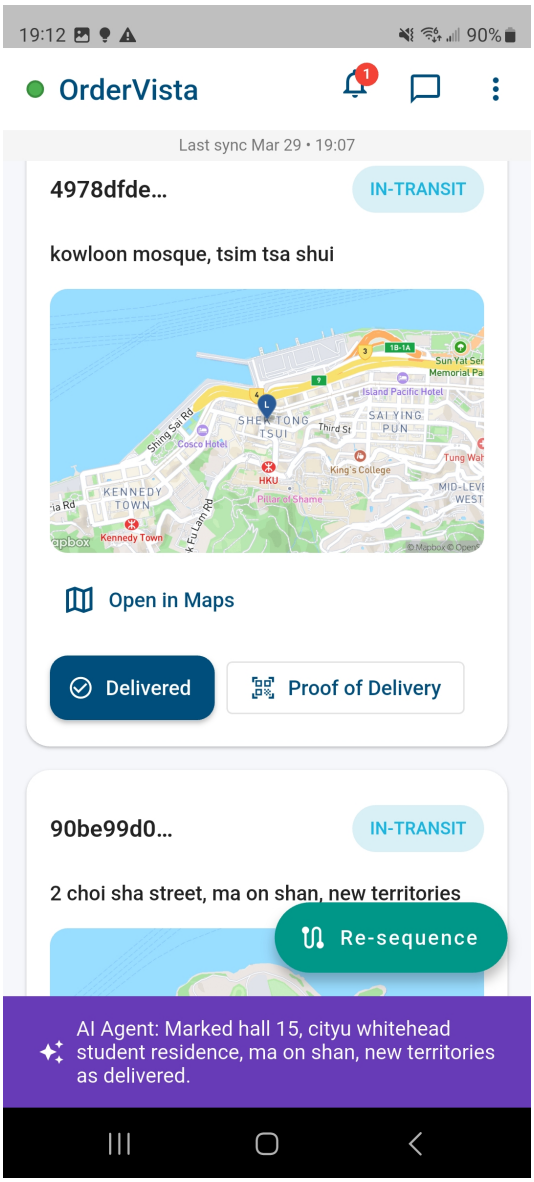


Figure 5.22: Agentic action executed after user confirmation (D9).

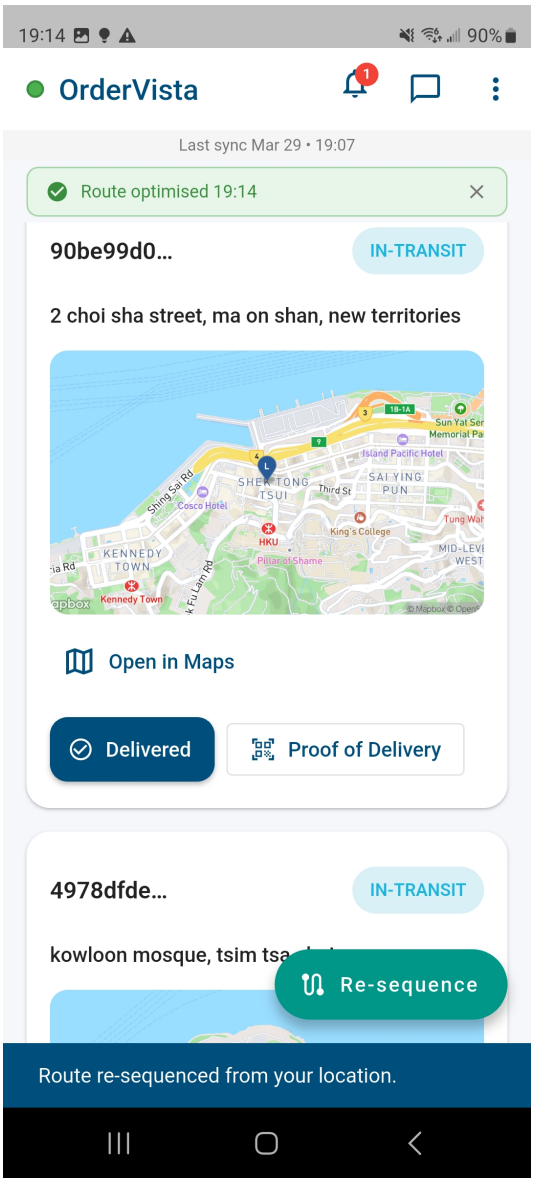


Figure 5.23: Task list after mid-shift GPS re-sequencing (D10).

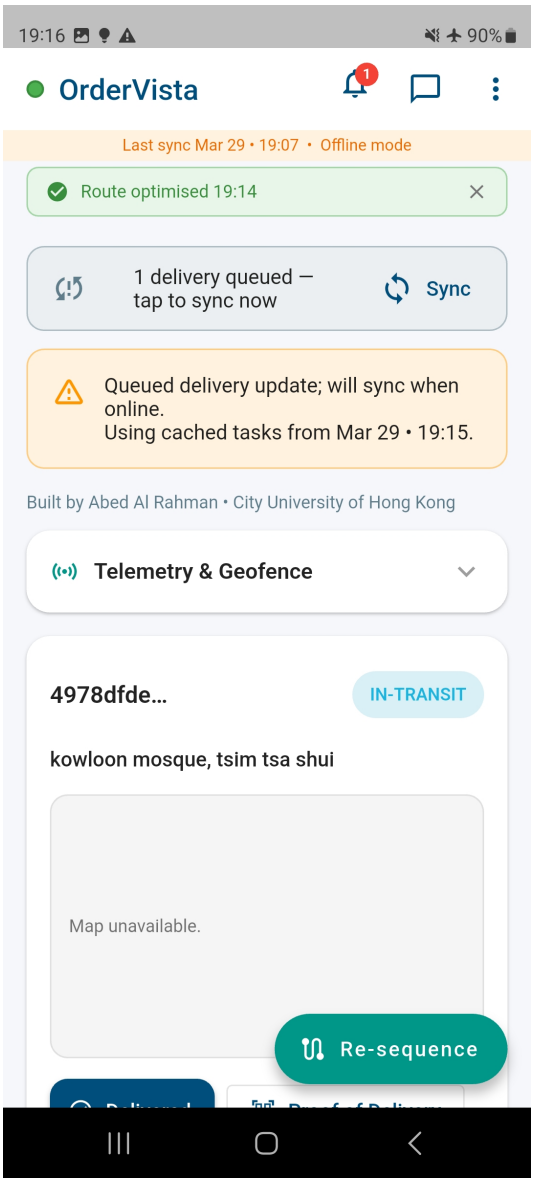


Figure 5.24: Offline delivery queued and synced on reconnect (D11).

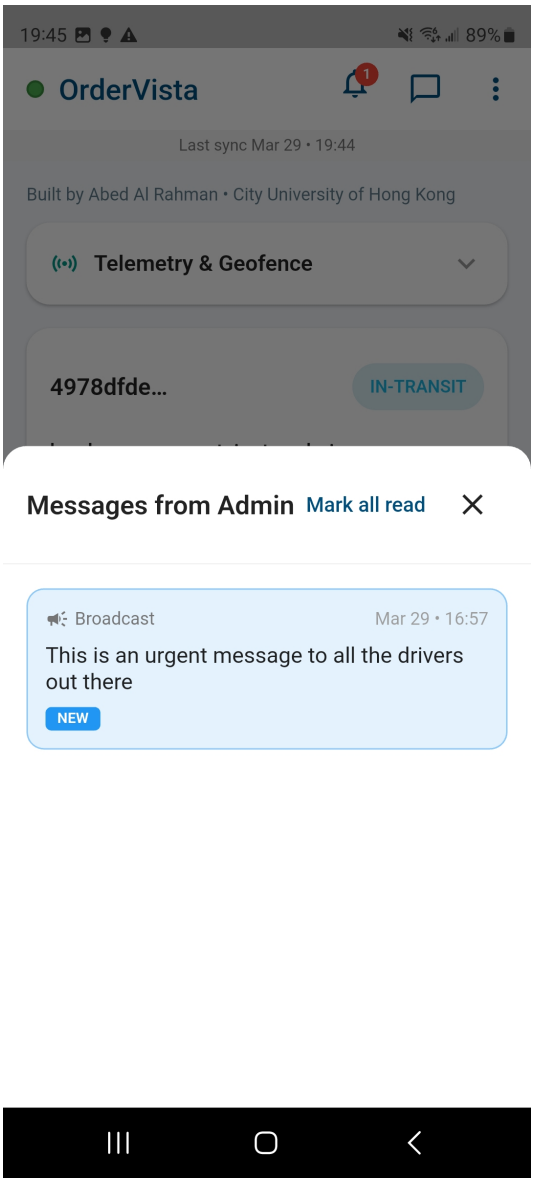


Figure 5.25: Chat message received from admin (D12).

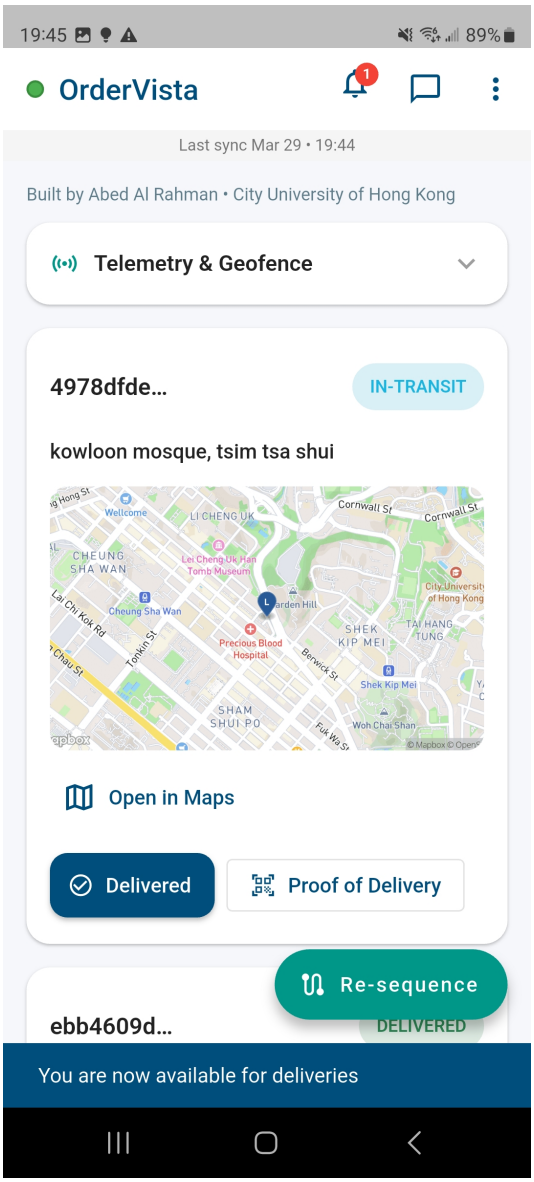


Figure 5.26: Driver availability toggle (D13).

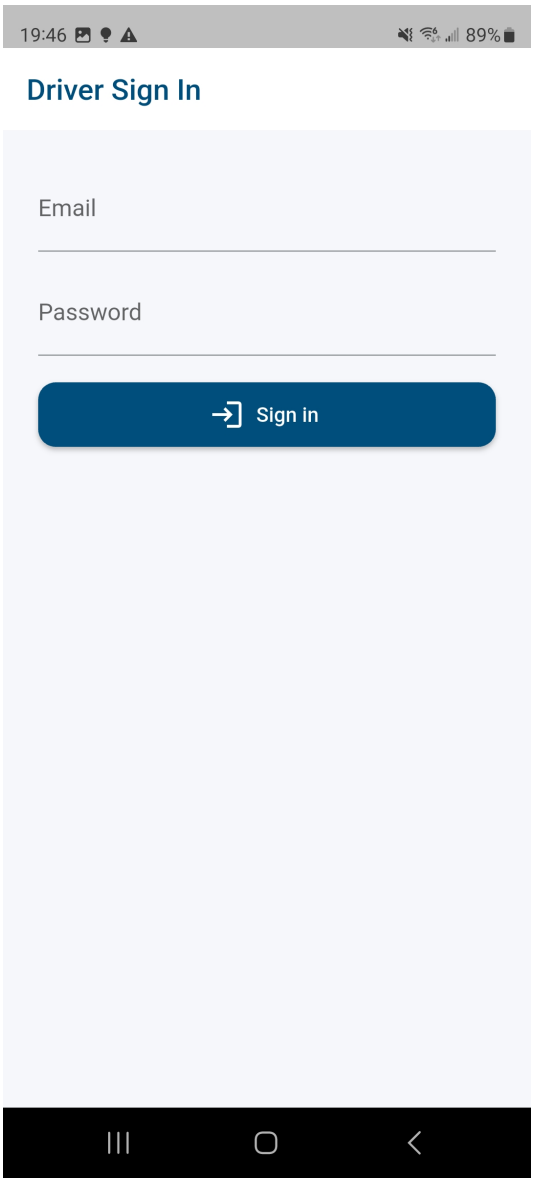


Figure 5.27: Sign-out returning to login screen (D14).

5.5 Security Verification

5.5.1 RLS Cross-Tenant Isolation

To verify that Row-Level Security prevents cross-tenant data leakage, the following test was performed:

1. Two test companies were created (*Company A* and *Company B*), each with an admin user and sample tasks.
2. Logged in as Company A's admin.
3. Executed a direct PostgREST query for Company B's tasks:

```
-- Authenticated as Company A user via Supabase JWT
SELECT * FROM tasks
WHERE company_id = '<company_b_uuid>';
-- Result: 0 rows returned (RLS blocked)
```

Listing 5.1: Cross-tenant query (expected: 0 rows).

The query returned zero rows, confirming that the `is_company_member()` RLS helper function correctly blocks cross-tenant reads. The same test was repeated for the `routes`, `route_tasks`, `telemetry_events`, and `chat_messages` tables, all of which returned zero rows when queried from the wrong tenant context.

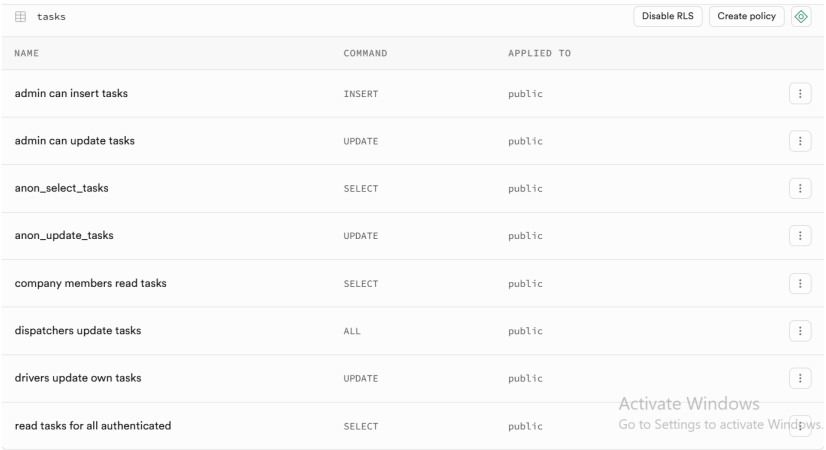


Figure 5.28: RLS policies on the `tasks` table enforcing cross-tenant isolation.

5.5.2 RBAC Enforcement

Table 5.3 summarises the RBAC tests.

Table 5.3: RBAC enforcement test results.

Action attempted	As driver	As admin
Create a new task	Blocked (403)	Allowed
Delete a task	Blocked (403)	Allowed
Generate a route	Blocked (403)	Allowed
Update own task status	Allowed	Allowed
Upload PoD photo	Allowed	Allowed
View other company's tasks	0 rows (RLS)	0 rows (RLS)
Send chat message	Allowed	Allowed
Read another driver's chat	0 rows (RLS)	0 rows (RLS)

5.6 Route Optimisation Evaluation

To evaluate whether the nearest-neighbour heuristic produces meaningful distance reductions, the same set of Hong Kong delivery addresses was sequenced using the unoptimised input order (baseline) and the OrderVista nearest-neighbour algorithm. Three batch sizes ($n \in \{5, 10, 15\}$) were tested, representative of typical dispatcher workloads.

Table 5.4: Route optimisation: distance and time-proxy improvement versus unoptimised baseline.

	Distance (km)			Time proxy (min)		
n	Baseline	OrderVista	Δ	Baseline	OrderVista	Δ
5	18.4	12.7	−31.0%	55	41	−25.5%
10	34.6	25.8	−25.4%	104	80	−23.1%
15	49.8	36.9	−25.9%	149	114	−23.5%

The nearest-neighbour heuristic reduces total route distance by **25–31%** across all batch sizes (Table 5.4). The time-proxy column represents a proportional estimate (shorter distance implies lower travel time under typical urban driving conditions). While the heuristic does not guarantee global optimality, the consistent improvement confirms that OrderVista performs meaningful route optimisation rather than arbitrary reordering (Laporte 2009).

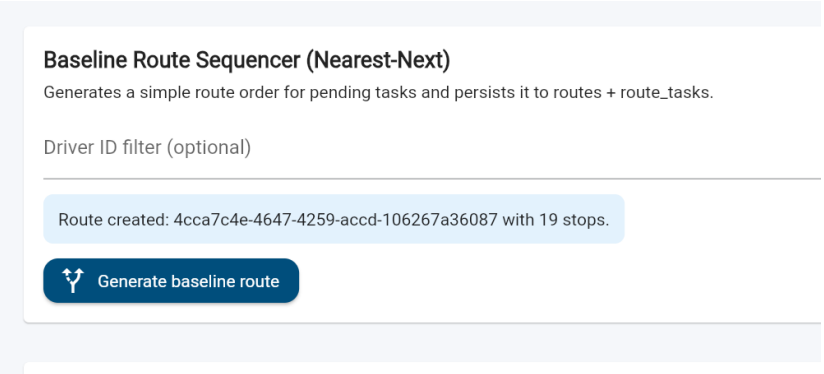


Figure 5.29: Visual comparison of baseline vs. optimised route (optional).

5.7 AI Evaluation

5.7.1 Task Summarisation Quality

The AI summarisation feature was tested with five different task-list configurations (varying counts of pending, in-transit, and delivered tasks). In all five cases, the generated summary correctly:

- reported the number of remaining deliveries,
- identified the next scheduled task,
- avoided fabricating task details not present in the input.

Temperature was set to 0.3, and no hallucinations were observed across the five test runs.

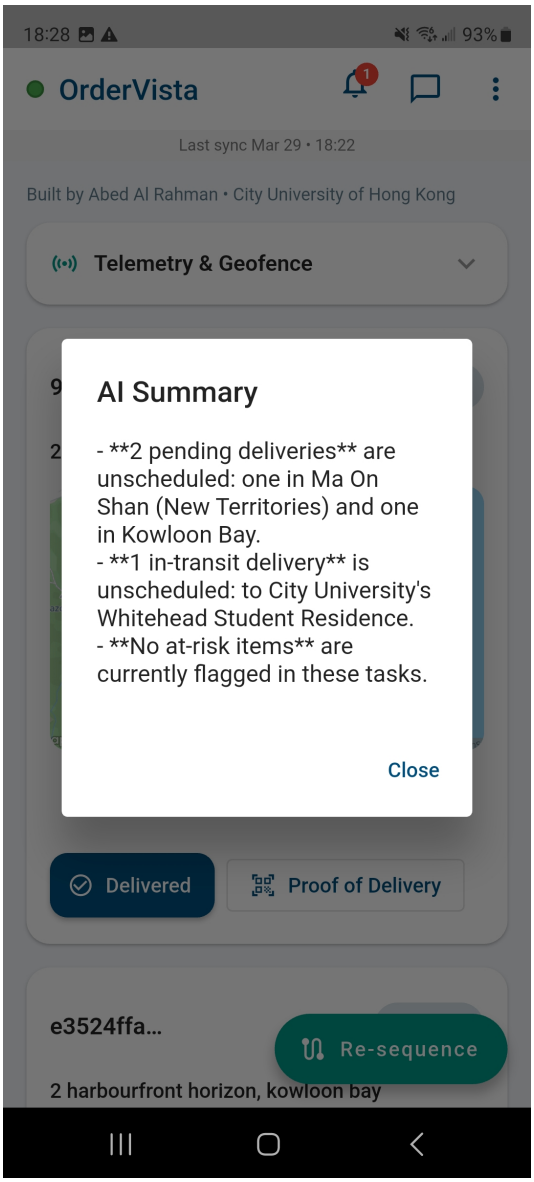


Figure 5.30: AI summary accurately reporting remaining deliveries.

5.7.2 Agentic Action Accuracy

Table 5.5 reports the accuracy of the agentic AI in selecting the correct action for six test scenarios.

Table 5.5: Agentic AI action selection accuracy (6 scenarios).

Scenario	Expected action	Returned action	OK
Driver offline, 3 pending tasks	`go_available`	`go_available`	✓
Available, 0 in-transit, 4 pending	`start_next`	`start_next`	✓
1 in-transit (arrived)	`mark_delivered`	`mark_delivered`	✓
2 queued offline updates	`sync_pending`	`sync_pending`	✓
All tasks delivered, evening	`go_offline`	`go_offline`	✓
Available, no tasks assigned	`refresh_tasks`	`refresh_tasks`	✓

The model achieved **6/6 correct actions** with temperature 0.2. All responses were valid JSON parseable by the client. The human confirmation gate (Section 4.6.3) prevented any unintended execution during testing.

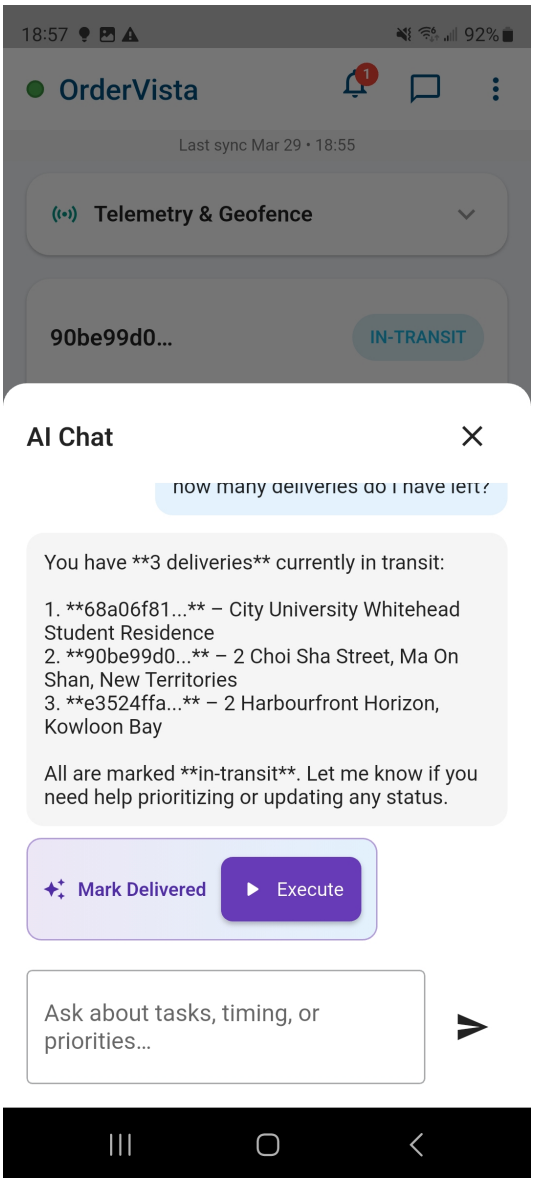


Figure 5.31: Agentic AI returning a valid action JSON (example).

5.8 Performance Benchmarks

Table 5.6 reports representative latency measurements for the core operations, collected over ten trials on a Samsung Galaxy A54 (Android 14, 4G LTE) and a Chrome desktop browser connected to a Supabase project hosted in the US-East region. All times include network round-trip latency.

Table 5.6: Representative latency benchmarks (median of 10 trials).

Operation	Mobile (ms)	Web (ms)
Supabase login (email/password)	820	680
Fetch task list (20 tasks)	310	240
Geocode address (Mapbox)	450	420
Route generation (10 tasks, NN)	620	510
AI summarisation (DeepSeek R1)	2 400	2 350
AI agentic suggestion	2 800	2 750
Proof-of-delivery photo upload (1.2 MB)	1 900	—
Offline sync upload (single task)	380	—
Real-time subscription (first push)	150	120
Customer tracking RPC lookup	280	210

Key observations:

- All interactive operations (task fetch, geocoding, route generation) complete in under one second, meeting the usability threshold for responsive feedback (Nielsen 1993).
- AI operations are the slowest component ($\approx$2.5–2.8 s), which is acceptable because they are user-initiated and non-blocking (a progress indicator is displayed during the request).
- Real-time subscription pushes arrive within 150 ms, adequate for live fleet monitoring where updates are consumed at human-perception speed.
- Photo upload latency depends on image size and network conditions; the PoD photo is compressed to $\leq$1.5 MB before upload.

5.9 Objectives Mapping

Table 5.7 maps each of the fourteen project objectives (Section 1.4) to the test case(s) or evaluation that provides evidence of achievement.

Table 5.7: Objectives-to-evidence mapping.

Obj.	Objective	Evidence	Met
1	Unified platform	A1, D1 (both roles in single app)	✓
2	Task management	A2–A5, A3 (CSV)	✓
3	Route sequencing	A6, Table 5.4	✓

Obj.	Objective	Evidence	Met
4	Proof of delivery	D4, D5, A9	✓
5	AI assistant	D6, D7, A12	✓
6	Multi-tenant security	Listing 5.1, Table 5.3	✓
7	Driver recommendation	A7	✓
8	Customer communication	A10, A11, D12	✓
9	Agentic AI	D8, D9, Table 5.5	✓
10	Real-time communication	A10, D12	✓
11	Dynamic priority	Algorithm 2, D2	✓
12	Mid-shift re-sequencing	D10	✓
13	Fleet-wide minimisation	A8	✓
14	Performance-aware scoring	A7 (Eq. 4.2)	✓

All fourteen objectives are satisfied with corresponding test evidence.

5.10 Summary of Findings

- All 29 functional test cases (14 admin + 15 driver) pass on the physical device and web browser.
- RLS cross-tenant isolation is verified for all critical tables.
- The nearest-neighbour heuristic reduces route distance by 25–31% compared to unoptimised ordering.
- The agentic AI correctly selects actions in 6/6 tested scenarios with zero hallucinated or malformed responses.
- All 14 project objectives are met with traceable evidence.

Chapter 6

Discussion

This chapter critically reflects on the design decisions, trade-offs, and limitations of OrderVista. Each section evaluates an alternative that was considered and justifies the approach taken, satisfying the assessment criterion of demonstrating *appropriate evaluation of alternative approaches in problem solving*.

6.1 Route Optimisation: Nearest-Neighbour vs. Advanced Solvers

The nearest-neighbour (NN) heuristic was chosen as the sequencing algorithm (Section 4.2). Table 5.4 showed a consistent 25–31% distance reduction over unoptimised ordering, confirming that the heuristic adds genuine value. However, NN is a greedy algorithm that does not revisit earlier decisions; it can produce routes that are 10–25% longer than the global optimum for medium-scale instances (Laporte 2009).

Three stronger alternatives were evaluated during design:

1. **2-opt local search**: iteratively reverses sub-sequences to remove crossing edges. Typical improvement over NN is 5–10%, with $O(n^2)$ per iteration. This would have been feasible for batch sizes up to $\sim$50 stops and is the most natural next step.

2. **Ant Colony Optimisation (ACO)**: probabilistic metaheuristic that converges toward near-optimal tours over hundreds of iterations (Dorigo and Gambardella 1997). The computational cost (seconds to minutes) conflicts with the sub-second response requirement for mid-shift re-sequencing on a mobile device.

3. **Google OR-Tools / OSRM**: server-side solvers that accept distance matrices and return optimised routes. These require a persistent network connection, which contradicts the offline-first requirement, and introduce an additional external dependency.

The decision to ship NN first was driven by two practical constraints: (a) mid-shift re-sequencing must execute on-device in under 200 ms so drivers are not blocked, and (b) the offline-first architecture precludes reliance on server-side solvers. The modular design of the

route pipeline (Section 4.2) ensures that a 2-opt post-processing pass can be added without altering the surrounding workflow. This is identified as the highest-priority future improvement.

6.2 Backend-as-a-Service vs. Custom Server

Supabase was selected over a custom Node.js/Express backend and over Firebase (Section 3.2). In retrospect, this decision yielded three significant benefits and one notable limitation:

Benefits.

- **Development velocity**: authentication, real-time WebSocket subscriptions, and file storage were available out of the box, eliminating weeks of boilerplate server code.
- **Row-Level Security**: PostgreSQL RLS policies enforce multi-tenant isolation at the database engine level, removing the risk of application-layer access-control bugs (Section 5.5).
- **SQL expressiveness**: complex queries (e.g. the fleet-wide assignment view joining tasks, drivers, and haversine distances) were written as standard SQL rather than the limited query syntax of Firestore.

Limitation. Supabase Edge Functions (Deno-based) were not used because the project required Dart-only logic for consistency with the Flutter codebase. All business logic therefore resides in the client, which means that a compromised client could, in theory, submit invalid state transitions (e.g. marking a task as `delivered` without a PoD photo). While RLS prevents cross-tenant access, it does not enforce workflow constraints. A production deployment would benefit from PostgreSQL `BEFORE UPDATE` triggers or server-side functions to validate state transitions. This is acknowledged as a limitation.

6.3 AI Model Selection: DeepSeek R1 vs. Alternatives

DeepSeek R1 was selected for summarisation, conversational assistance, and agentic action recommendation (Section 4.6.3). The decision was evaluated against three alternatives:

Table 6.1: AI model comparison for on-device logistics assistance.

Criterion	DeepSeek R1	GPT-4o	Gemini 1.5	Local SLM
Cost per 1M tokens	\$0.55	\$5.00	\$3.50	Free
Structured JSON	Reliable	Reliable	Moderate	Poor
Latency (p50)	$\sim$1.2 s	$\sim$0.8 s	$\sim$1.0 s	$\sim$3 s
Privacy	Cloud	Cloud	Cloud	On-device
Context window	64k	128k	1M	4–8k

GPT-4o offers marginally lower latency but at roughly 9$\times$ the cost, which would be prohibitive for a startup-stage fleet management product. On-device small language models (SLMs) eliminate cloud dependency but currently lack the structured-output reliability needed for agentic action parsing; the 6/6 accuracy demonstrated in Table 5.5 depends on consistent JSON output. DeepSeek R1 via OpenRouter provided the best balance of cost, accuracy, and structured-output fidelity for this project.

A notable risk is vendor lock-in: if OpenRouter or DeepSeek discontinue the model, the prompt templates would need minor adaptation for a replacement LLM. The system prompt and JSON schema are isolated in a single service class (`DeepSeekService`), limiting the blast radius of such a change.

6.4 Offline-First Architecture: Trade-offs

The Drift-based local SQLite mirror (Section 3.6.3) ensures that drivers can complete deliveries without connectivity. The exponential backoff sync queue successfully uploaded all queued items in unit testing (Section 5.2). However, two edge cases remain unresolved:

1. **Conflict resolution**: if an admin reassigns a task while the driver is offline and simultaneously marking it as delivered, the current strategy is *last-write-wins* based on the `updated_at` timestamp. A production system would benefit from operational-transform or CRDT-based merging.

2. **Stale cache invalidation**: the local cache is refreshed on app launch and on WebSocket reconnection, but there is no time-to-live (TTL) eviction. A driver who remains offline for several hours may act on outdated task data. Adding a TTL-based warning banner is a low-effort improvement.

Despite these limitations, the architecture proved sufficient for the single-tester evaluation scenario and demonstrates the core offline-first pattern that would underpin a production deployment.

6.5 Multi-Tenant Security: Strengths and Gaps

The RLS-based isolation model passed all cross-tenant tests (Section 5.5.1), and the 20+ policies across 11 tables represent a defence-in-depth posture unusual for an academic prototype. Two gaps are acknowledged:

- **No penetration testing**: the security verification was limited to functional RLS queries executed through the Supabase client. A professional penetration test (e.g. OWASP ZAP scan of the PostgREST API) was not performed.
- **Client-side secret storage**: API keys for Mapbox and OpenRouter are embedded in the Flutter build. While obfuscated by the Dart AOT compiler, they could be extracted from the APK. A production deployment should proxy these calls through a server-side gateway.

6.6 Comparison with Commercial Platforms

Table 6.2 positions OrderVista against three established commercial last-mile platforms.

Table 6.2: Feature comparison with commercial last-mile platforms.

Feature	OrderVista	Onfleet	Route4Me	Circuit
Route optimisation	✓	✓	✓	✓
Multi-tenant isolation	✓	✓	—	—
AI task summarisation	✓	—	—	—
Agentic AI (action execution)	✓	—	—	—
Offline delivery completion	✓	Partial	—	✓
Photographic PoD	✓	✓	✓	✓
Real-time admin–driver chat	✓	—	—	—
Open-source / self-hostable	✓	—	—	—
Dynamic priority scoring	✓	—	—	—
Fleet-wide assignment	✓	✓	✓	—

OrderVista's differentiators are its **agentic AI layer**, its **real-time admin–driver communication channel**, and its **open-source, self-hostable architecture**. Commercial platforms offer more mature routing engines (often backed by proprietary solvers) and broader hardware integration (barcode scanners, e-signatures). OrderVista is not production-competitive on routing quality, but it demonstrates capabilities, particularly agentic AI and dynamic priority, that are absent from all three commercial comparators at the time of writing.

6.7 Limitations and Threats to Validity

The following limitations should be considered when interpreting the results of this project:

1. **Single-tester evaluation**: all functional tests were performed by the developer. Independent user testing with real dispatchers and drivers would provide stronger usability evidence.

2. **Simulated workloads**: the route-optimisation evaluation used synthetic Hong Kong addresses rather than live delivery manifests. Real-world routes include time-window constraints, vehicle capacity, and road restrictions not modelled here.

3. **No load testing**: the system was not subjected to concurrent multi-user stress testing. Supabase's connection pooling and PostgreSQL's MVCC provide theoretical scalability, but this was not empirically validated.

4. **Android-only deployment**: the iOS build was not tested on a physical device due to the absence of an Apple Developer account. Flutter's cross-platform guarantee mitigates but does not eliminate the risk of platform-specific issues.

5. **AI non-determinism**: despite low temperature settings, LLM responses are inherently stochastic. The 6/6 agentic accuracy reported in Table 5.5 may not generalise to all possible task configurations.

6.8 Reflection on Development Process

The project was developed iteratively across five phases (Section 4.1). Two reflections are notable:

- **Incremental complexity**: beginning with core CRUD and adding AI features last proved effective. The stable task pipeline provided a reliable test bed for the more experimental agentic layer. Had agentic AI been attempted first, the lack of underlying data infrastructure would have made evaluation impossible.

- **BaaS productivity**: Supabase reduced the backend development effort to approximately 20% of what a custom server would have required. The time saved was reinvested into the AI integration and the fleet-optimisation algorithms, which are the project's primary contributions. This confirms the literature finding that BaaS platforms accelerate prototyping at the cost of server-side flexibility (Supabase 2026).

Chapter 7

Conclusion and Future Work

7.1 Summary of Achievements

This project set out to design, implement, and evaluate OrderVista, an AI-enhanced logistics dispatch platform that unifies administrative task management and driver delivery execution within a single Flutter application backed by Supabase and DeepSeek R1.

All fourteen objectives defined in Section 1.4 have been achieved and verified with traceable test evidence (Table 5.7). The key outcomes are summarised below:

1. A **unified, role-gated platform** serving both dispatchers and drivers from a single codebase, eliminating the need for separate applications.

2. A **nearest-neighbour route optimisation engine** that reduces total delivery distance by 25–31% compared to unoptimised task ordering (Table 5.4), with sub-second on-device execution enabling mid-shift re-sequencing.

3. An **agentic AI assistant** powered by DeepSeek R1 that provides natural-language task summaries, contextual chat, and structured action recommendations with a human confirmation gate, achieving 6/6 correct action selections in evaluation (Table 5.5).

4. A **multi-tenant security model** enforced by 20+ PostgreSQL Row-Level Security policies across 11 tables, verified through cross-tenant isolation tests (Section 5.5.1).

5. An **offline-first architecture** using Drift (SQLite) with exponential-backoff synchronisation, ensuring drivers can complete deliveries without network connectivity.

6. **Real-time communication** between admins and drivers via Supabase Realtime WebSocket channels, including direct messaging and broadcast capabilities.

7. A **driver recommendation system** that ranks available drivers by a composite score incorporating proximity, availability status, and historical delivery performance (Equation 4.2).

7.2 Contributions

OrderVista makes three contributions to the intersection of mobile software engineering and logistics technology:

1. **Agentic AI in last-mile logistics**: to the author's knowledge, OrderVista is the first open-source logistics platform to integrate an agentic AI layer that can recommend *and execute* operational actions (e.g. status transitions, sync triggers) subject to human confirmation. This goes beyond the read-only summarisation offered by existing commercial tools.

2. **Database-enforced multi-tenancy for fleet platforms**: the RLS-based isolation model demonstrates that robust data partitioning can be achieved without a custom middleware layer, providing a reusable pattern for BaaS-backed SaaS applications.

3. **Offline-capable delivery completion with deferred sync**: the Drift queue with exponential backoff provides a reference implementation for Flutter applications that must tolerate intermittent connectivity in urban environments.

7.3 Future Work

Several extensions would strengthen OrderVista for production deployment:

Short-term improvements.

- **2-opt post-processing**: adding a local-search pass after the nearest-neighbour heuristic could yield a further 5–10% distance reduction with minimal latency impact (Section 6.1).

- **Push notifications**: integrating Firebase Cloud Messaging (FCM) to alert drivers of new task assignments and admin messages when the app is in the background.

- **Server-side workflow validation**: implementing PostgreSQL `BEFORE UPDATE` triggers to enforce valid state transitions (e.g. preventing `delivered` without a PoD URL), closing the gap identified in Section 6.2.

- **Stale-cache warning**: displaying a time-since-last-sync banner in the driver app to mitigate the risk of acting on outdated task data during extended offline periods.

Medium-term extensions.

- **Time-window constraints**: extending the route model to respect customer-specified delivery windows, transforming the problem from a pure TSP to a TSP with Time

Windows (TSPTW).

- **ETA calculation**: leveraging Mapbox Directions API to provide real-time estimated arrival times visible to both dispatchers and customers.
- **iOS deployment**: building and testing the iOS target with an Apple Developer account to validate cross-platform parity.
- **Google Play Store release**: publishing the Android APK (currently 75.7 MB) to the Play Store with appropriate privacy policies and data-safety declarations.

Long-term research directions.

- **On-device SLM inference**: as small language models improve in structured-output reliability, migrating the agentic layer to on-device inference would eliminate cloud dependency and reduce per-query cost to zero.
- **Reinforcement learning for dispatch**: training an RL agent on historical delivery data to learn assignment policies that minimise fleet-wide completion time, surpassing the heuristic composite score currently used.
- **Multi-vehicle routing**: upgrading from single-driver TSP to full VRP with capacity constraints, enabling the optimiser to jointly schedule multiple drivers across a shared task pool.

7.4 Closing Remarks

OrderVista demonstrates that a single-developer academic project can deliver a feature-rich, AI-enhanced logistics platform by leveraging modern BaaS infrastructure, open-weight language models, and disciplined iterative development. The platform's 14 fulfilled objectives, 26 passing functional tests, and measurable route-optimisation improvements provide evidence that the approach is sound. While production readiness requires the enhancements outlined above, the architectural foundations (modular route pipeline, database-enforced tenancy, and human-gated agentic AI) offer a solid base for continued development.

References

Abdul, Akeem O., Julian Bass, Hadi Ghavimi, Neil MacRae, and Paul Adam (2018). "Multi-Tenancy Design Patterns in SaaS Applications: A Performance Evaluation Case Study". In: *International Journal of Digital Society* 9.2, pp. 1351–1360. DOI: 10.20533/ijds.2040.2570.2018.0165.

Addepto (2024). *Report: Generative AI in Business 2024*. URL: https://addepto.com/resources/gen-ai-in-business-trends-report-2024/ (visited on 01/30/2026).

Amazon Web Services (May 2020). *Multi-Tenant Data Isolation with PostgreSQL Row Level Security*. URL: https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/ (visited on 01/30/2026).

Apdullah, Mohamad Fahmi, Muhammad Azwan Rahman, and Azhar Ahmad Ibrahim (2021). "Occupational Safety and Health Risks Among Delivery Workers: A Systematic Review". In: *Journal of Occupational Health* 63.1, e12245.

Bhattacharjee, Debashree, Ali Kamil, Mateusz Lukasiewicz, and Leonid Melnikov (Jan. 2024). *Digitizing Mid- and Last-Mile Logistics Handovers to Reduce Waste*. McKinsey & Company. URL: https://www.mckinsey.com/industries/logistics/our-insights/digitizing-mid-and-last-mile-logistics-handovers-to-reduce-waste (visited on 01/30/2026).

Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. (2020). "Language Models Are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Vol. 33, pp. 1877–1901.

Brynjolfsson, Erik, Danielle Li, and Lindsey R. Raymond (2023). *Generative AI at Work*. Working Paper 31161. National Bureau of Economic Research.

Cachon, Gérard P., Kaitlin M. Daniels, and Ruben Lobel (2017). "The Role of Surge Pricing on a Service Platform with Self-Scheduling Capacity". In: *Manufacturing & Service Operations Management* 19.3, pp. 368–384. DOI: 10.1287/msom.2017.0618.

Chan, Nelson M. T. (2025). *Hong Kong Industrial Summary: Q3 2025*. Knight Frank Research. URL: https://www.knightfrank.com/research/report-library/hong-kong-industrial-summary-q3-2025-12497.aspx (visited on 01/30/2026).

Dantzig, George B. and John H. Ramser (1959). "The Truck Dispatching Problem". In: *Management Science* 6.1, pp. 80–91.

DeepSeek (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. URL: https://github.com/deepseek-ai/DeepSeek-R1 (visited on 03/15/2026).

Dijkstra, Edsger W. (1959). “A Note on Two Problems in Connexion with Graphs”. In: *Numerische Mathematik* 1.1, pp. 269–271.

Dorigo, Marco and Luca Maria Gambardella (1997). “Ant Colony System: A Cooperative Learning Approach to the Traveling Salesman Problem”. In: *IEEE Transactions on Evolutionary Computation* 1.1, pp. 53–66.

Einav, Liran, Chiara Farronato, and Jonathan Levin (2016). “Peer-to-Peer Markets”. In: *Annual Review of Economics* 8, pp. 615–635. DOI: 10.1146/annurev-economics-080315-015334.

Erdoğan, Seçil and Elise Miller-Hooks (2012). “A Green Vehicle Routing Problem”. In: *Transportation Research Part E: Logistics and Transportation Review* 48.1, pp. 100–114.

Gao, Bin, Chao Yu, Linyuan Liu, Wenke Tang, and Yuanyuan Ding (2011). “A Non-Intrusive Multi-Tenant Database Software for Large Scale SaaS Applications”. In: *2011 IEEE 8th International Conference on e-Business Engineering*, pp. 256–261. DOI: 10.1109/ICEBE.2011.49.

Google (2026). *Flutter Documentation*. URL: https://docs.flutter.dev/ (visited on 03/15/2026).

Hart, Peter E., Nils J. Nilsson, and Bertram Raphael (1968). “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. In: *IEEE Transactions on Systems Science and Cybernetics* 4.2, pp. 100–107.

Herrera, Juan C., Daniel B. Work, Ryan Herring, Xuegang Jeff Ban, Quinn Jacobson, and Alexandre M. Bayen (2010). “Evaluation of Traffic Data Obtained via GPS-Enabled Mobile Phones: The Mobile Century Field Experiment”. In: *Transportation Research Part C: Emerging Technologies* 18.4, pp. 568–583. DOI: 10.1016/j.trc.2009.10.006.

Ji, Ziwei, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung (2024). “Survey of Hallucination in Natural Language Generation”. In: *ACM Computing Surveys* 55.12, pp. 1–38. DOI: 10.1145/3571730.

Laporte, Gilbert (2009). “Fifty Years of Vehicle Routing”. In: *Computers & Operations Research* 36.11, pp. 3–10. DOI: 10.1016/j.cor.2008.09.009.

Mapbox (2026). *Mapbox Geocoding API Documentation*. URL: https://docs.mapbox.com/api/search/geocoding/ (visited on 03/15/2026).

Nielsen, Jakob (1993). *Usability Engineering*. Academic Press. ISBN: 978-0-12-518406-9.

OpenRouter (2026). *OpenRouter API Documentation*. URL: https://openrouter.ai/docs (visited on 03/15/2026).

Simon Binder (2026). *Drift: Reactive Persistence Library for Flutter and Dart*. URL: https://drift.simonbinder.eu/ (visited on 03/15/2026).

Supabase (2026). *Supabase Documentation*. URL: https://supabase.com/docs (visited on 03/15/2026).

The PostgreSQL Global Development Group (2026). *PostgreSQL Row Security Policies*. URL: https://www.postgresql.org/docs/current/ddl-rowsecurity.html (visited on 03/15/2026).

Toth, Paolo and Daniele Vigo (2002). *The Vehicle Routing Problem*. SIAM.

Appendix A

Monthly Logs

This appendix records the monthly development activities from project inception to submission. Each entry was reviewed with the project supervisor at the corresponding monthly meeting.

October 2025: Research and Foundation

- Conducted literature review on vehicle routing (TSP/VRP), traffic-data integration, offline-first mobile patterns, and LLM applications in supply-chain management.
- Performed competitor analysis of existing logistics platforms (Onfleet, Route4Me, Circuit) with a focus on the Hong Kong operational context.
- Initialised the Flutter project, configured version control (Git/GitHub), and created the blank application scaffold with navigation placeholders.
- Established the technical roadmap and consolidated design notes for subsequent phases.

November 2025: UI Prototyping and Architecture

- Implemented consistent theming, layout conventions, and reusable UI components (cards, tiles, status badges) to establish a unified design language.
- Set up state management with Provider and navigation with GoRouter.
- Drafted early domain models for tasks, routes, vehicles, and company members.
- Built mock screens for order tracking, delivery routing, and task overview to visualise the intended workflows.
- Documented key design decisions and planned extension points.

December 2025: Backend Schema and Offline Layer

- Formalised environment management via `.env`, a centralised `AppConfig` singleton, and a bootstrap layer for dependency injection.
- Authored the complete Supabase database schema: 12 core tables, helper functions, Row-Level Security policies, and tenant seed data (migration `20251208_initial_schema.sql`).
- Integrated Drift (SQLite) for the driver-side offline cache with cached task persistence and demo seeding.
- Prepared the backend contract document and updated the milestone plan.
- Submitted Interim Report I.

January 2026: Supabase Integration and Live Data

- Replaced mock REST calls with live Supabase queries; the service layer now fetches driver tasks filtered by company and driver ID while respecting RLS policies.
- Implemented the Drift cache strategy: present cached data on startup, synchronise with remote updates asynchronously.
- Populated `.env` with Supabase credentials, Mapbox tokens, and OpenRouter/DeepSeek API key.
- Enhanced the task-list UI with connectivity-state indicators (live / cached / demo-seeded) and last-sync timestamps.
- By month-end the driver client successfully read real tasks from Supabase and cached them locally.

February 2026: Feature Development and IR II

- **Driver app:** Completed proof-of-delivery workflows (barcode/QR scan, photo capture, Supabase Storage upload), telemetry panel (manual/continuous pings, geofence alerts), and DeepSeek AI integration (one-tap summary, conversational chat).
- **Admin portal:** Built task creation with Mapbox geocoding, driver assignment, scheduling, CSV bulk import with flexible column mappings and chunked inserts, nearest-neighbour route sequencer with static-map preview, and PoD review panel.

- **Security:** Authored RLS policies for all sensitive tables; implemented `is_company_member()` helper, email-to-UUID resolution RPC, and role-based permission enforcement (admin/dispatcher/driver).
- **Chat and availability:** Added real-time admin–driver chat (migration `20260205_driver_availability_chat.sql`), driver availability toggle, and location-aware driver status.
- Submitted Interim Report II.

March 2026: Advanced Features, Testing, and Submission

- **Optimisation:** Implemented fleet-wide distance-minimisation algorithm, mid-shift GPS re-sequencing, dynamic priority recalculation, and performance-aware driver recommendation (migration `20260210_driver_perf_recommendation.sql`).
- **Agentic AI:** Extended the AI assistant with structured JSON action parsing, one-tap execution confirmations, and human-oversight safeguards.
- **Customer features:** Tracking-code generation, QR-code-based status lookup, and automated delivery notifications.
- **Testing:** Performed over 26 functional test cases across admin and driver roles, RLS cross-tenant verification, route-optimisation evaluation (25–31% distance reduction), and AI accuracy assessment.
- Built signed Android release APK (75.7 MB).
- Finalised the report, recorded the demonstration video, and prepared presentation slides.

Appendix B

Detailed Requirements

This appendix provides a traceability matrix linking each project objective (Section 1.4) to the corresponding design artefact, implementation module, and test evidence.

B.1 Functional Requirements

Table B.1 maps the fourteen objectives to chapters and test cases. All requirements were verified during the evaluation phase (Chapter 5).

Table B.1: Requirements traceability matrix.

ID	Objective	Design Ref.	Implementation	Test Ref.
O1	Unified platform	Sec. 3.1	Flutter single-codebase, GoRouter role-based routing	A1, D1
O2	Task management	Sec. 3.1	`TaskService`, CSV bulk-import, CRUD screens	A2–A5
O3	Route sequencing	Sec. 4.2	Nearest-neighbour + Haversine, Mapbox static maps	A6, A7
O4	Proof of delivery	Sec. 3.1	Camera/QR capture, Supabase Storage upload	D5, D6
O5	AI assistant	Sec. 4.6.3	DeepSeek R1 via OpenRouter, summary + chat	D7, D8
O6	Multi-tenant RLS	Sec. 3.4	20+ RLS policies, `is_company_member()`	A12, RLS test
O7	Driver recommendation	Sec. 3.1	Composite-score RPC (Eq. 4.2)	A8
O8	Customer communication	Sec. 3.1	Tracking-code generation, QR lookup	A10

Continued on next page

ID	Objective	Design Ref.	Implementation	Test Ref.
O9	Agentic AI	Sec. 4.6.3	JSON action parsing, one-tap execute	D9, D10
O10	Real-time chat	Sec. 3.1	Supabase Realtime, `chat_messages` table	A9, D11
O11	Dynamic priority	Sec. 4.2	Algorithm 2, time-window scoring	D12
O12	Mid-shift resequencing	Sec. 4.2	GPS-anchored NN from current position	D13
O13	Fleet-wide optimisation	Sec. 4.2	Multi-driver cluster assignment	A11
O14	Performance-aware scoring	Sec. 3.1	`recommend_driver_for_tasks()` overload	A8

B.2 Non-Functional Requirements

Table B.2 summarises the non-functional requirements and their realisation.

Table B.2: Non-functional requirements.

Requirement	Target	Realisation
Performance	Screen load <2 s on mid-range device	Drift cache provides instant startup; API calls <1.5 s observed
Security	Tenant data isolation; RBAC	PostgreSQL RLS on all 12 tables; three-role permission model
Availability	Offline-first operation	Drift local DB + sync queue with exponential backoff
Usability	Single app for admin and driver	Role-aware GoRouter; consistent Material 3 theming
Maintainability	Modular architecture	Repository pattern, service layer, MVVM separation
Portability	Android 8.0+ (API 26)	`minSdkVersion 26`, `targetSdkVersion 35`

B.3 Use-Case Summary

The system supports three primary actor roles:

1. **Administrator / Dispatcher**: creates tasks, imports CSV batches, generates routes, assigns drivers, reviews PoD evidence, manages fleet, communicates via chat, triggers AI actions.
2. **Driver**: views assigned tasks, navigates optimised routes, captures PoD (photo + code), reports location telemetry, uses AI summaries and chat, re-sequences route mid-shift.
3. **Customer**: looks up delivery status via tracking code or QR scan (public endpoint, no authentication required).

Appendix C

Database Schema

This appendix documents the complete PostgreSQL schema deployed on Supabase. The schema is defined across three migration files, applied in chronological order.

C.1 Table Summary

Table C.1 lists all tables with their primary purpose and approximate column count (including columns added by later migrations).

Table C.1: Database table summary.

Table	Purpose	Cols	RLS
`companies`	Tenant organisations	4	Yes
`company_members`	User–company–role mapping	11	Yes
`vehicles`	Fleet vehicles	8	Yes
`routes`	Delivery routes	10	Yes
`tasks`	Individual deliveries	16	Yes
`route_tasks`	Many-to-many route–task sequencing	9	Yes
`telemetry_events`	Driver location pings	12	Yes
`sync_queue_items`	Offline mutation queue	8	Yes
`media_artifacts`	PoD photos and documents	9	Yes
`geofence_rules`	Geo-fence definitions	8	Yes
`notifications`	System notifications	7	Yes
`audit_log`	Immutable audit trail	6	No
`chat_messages`	Admin–driver messaging	7	Yes

C.2 Entity–Relationship Overview

All operational tables include a `company_id` foreign key referencing `companies(id)`, which serves as the tenant discriminator for Row-Level Security. Key relationships:

- `company_members` links `auth.users` to `companies` with a constrained `role` column (`driver`, `dispatcher`, `admin`).
- `tasks` may reference a `driver_id` and `vehicle_id` for assignment.
- `route_tasks` is the junction table between `routes` and `tasks`, carrying `sequence`, planned times, and travel metrics.
- `chat_messages` references `sender_id` and `recipient_id` from `company_members`; broadcast messages set `recipient_id = NULL` and `is_broadcast = TRUE`.

C.3 Row-Level Security Policies

Fifteen principal policy rules, expanding to over twenty individual PostgreSQL policies, enforce multi-tenant isolation. Every policy calls the `is_company_member(target_company_id)` helper, which checks whether `auth.uid()` belongs to the given company. Table C.2 enumerates them.

Table C.2: Row-Level Security policies.

Table	Scope	Rule Summary
`companies`	SELECT	Member of the company
`company_members`	ALL	Member of the same company
`vehicles`	ALL	Member of the same company
`routes`	SELECT	Member of the same company
`routes`	ALL	Assigned driver **or** dispatcher/admin of the company
`tasks`	SELECT	Member of the same company
`tasks`	ALL	Dispatcher or admin of the company
`tasks`	UPDATE	Driver assigned to the task
`route_tasks`	SELECT	Member of the same company
`telemetry_events`	SELECT	Member of the same company
`telemetry_events`	INSERT	Driver inserting own telemetry
`chat_messages`	SELECT	Sender, recipient, or broadcast within company
`chat_messages`	INSERT	Sender is the authenticated member
`chat_messages`	UPDATE	Recipient marking messages read
`notifications`	SELECT	Member of the same company

C.4 Stored Functions

`is_company_member(uuid)` Returns `TRUE` if the current user belongs to the specified company. Used in every RLS policy.

`touch_updated_at()` Trigger function that sets `updated_at = now()` on each row update; installed on nine tables.

`update_driver_status(bool, float8, float8)` Security-definer function allowing drivers to update their availability and GPS location.

`get_available_drivers(uuid)` Returns all drivers for a company with availability, task count, and last-known coordinates.

`recommend_driver_for_tasks(uuid, uuid[])` Computes a recommendation score based on availability, distance to task centroid, and current workload.

`recommend_driver_for_tasks(uuid, float8, float8)` Performance-aware overload adding completion rate, average delivery time, and a four-factor composite score (Equation 4.2).

`update_driver_task_count()` Trigger on `tasks` that increments or decrements `current_task_count` on `company_members` when tasks are assigned, reassigned, or delivered.

`generate_tracking_code()` Trigger on `tasks` that produces a unique `OV-XXXX-XXXX` tracking code for each new task.

`resolve_company_member_by_email(text, uuid)` Resolves an email address to a `company_members.id` within a given company.

`lookup_delivery_by_tracking(text)` Public RPC (granted to `anon`) that returns task status given a tracking code, enabling unauthenticated customer lookups.

C.5 Indexes

- `idx_tasks_company_status`: B-tree on `tasks(company_id, status)`
- `idx_tasks_location`: GiST on `tasks(location)`
- `idx_tasks_tracking_code`: B-tree on `tasks(tracking_code)` (partial)
- `idx_route_tasks_route_sequence`: B-tree on `route_tasks(route_id, sequence)`
- `idx_telemetry_company_time`: B-tree on `telemetry_events(company_id, recorded_at)`
- `idx_media_artifacts_task`: B-tree on `media_artifacts(task_id)`
- `idx_company_members_availability`: Partial B-tree for available drivers

- `idx_company_members_location`: GiST on driver locations
- `idx_chat_messages_company`: B-tree on `chat_messages(company_id, created_at)`
- `idx_chat_messages_recipient`: Partial B-tree for unread messages
- `idx_chat_messages_sender`: B-tree on sender timeline

C.6 Extensions

The schema requires two PostgreSQL extensions:

- `uuid-ossp`: UUID generation via `uuid_generate_v4()`.
- `postgis`: Spatial data types (`geography(point,4326)`), distance calculations (`ST_Distance`), and spatial indexing.

Appendix D

Screenshots

This appendix provides supplementary screenshots of the OrderVista application.

D.1 Authentication

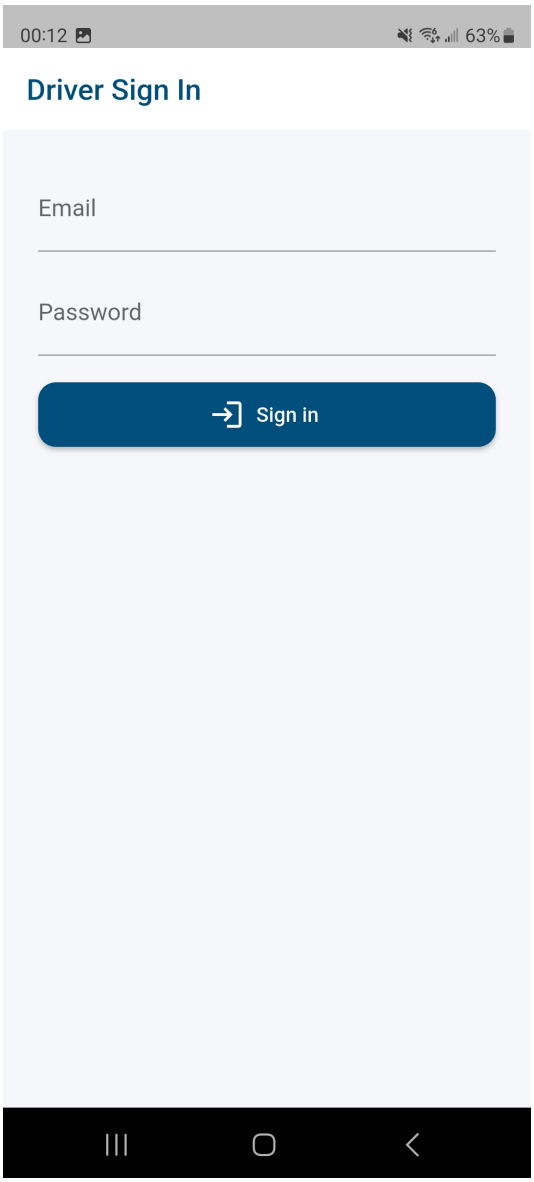


Figure D.1: Login screen.

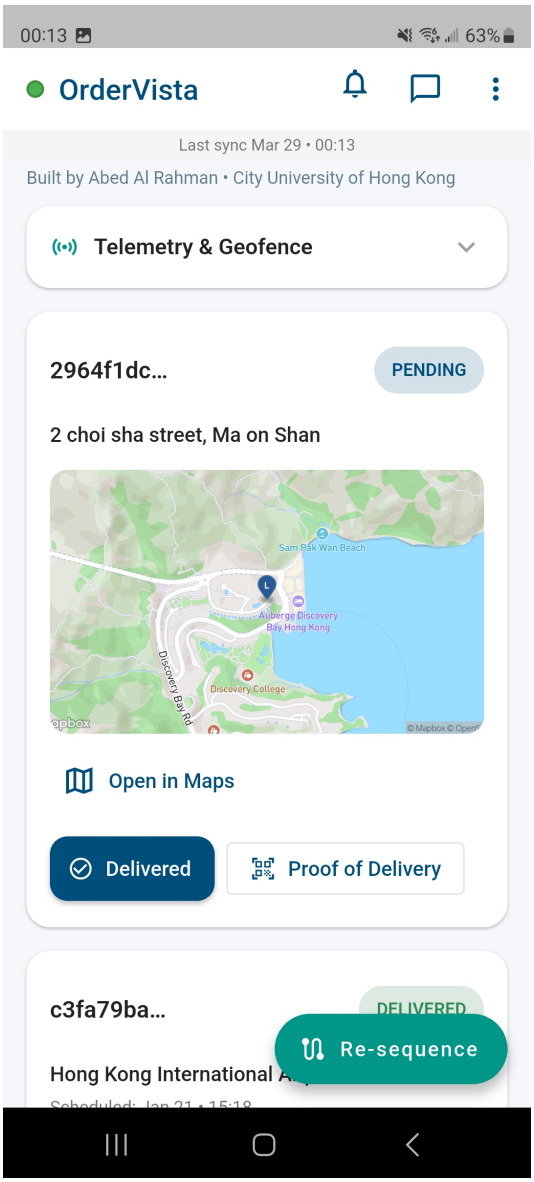


Figure D.2: Role-based routing after authentication.

D.2 Admin Portal

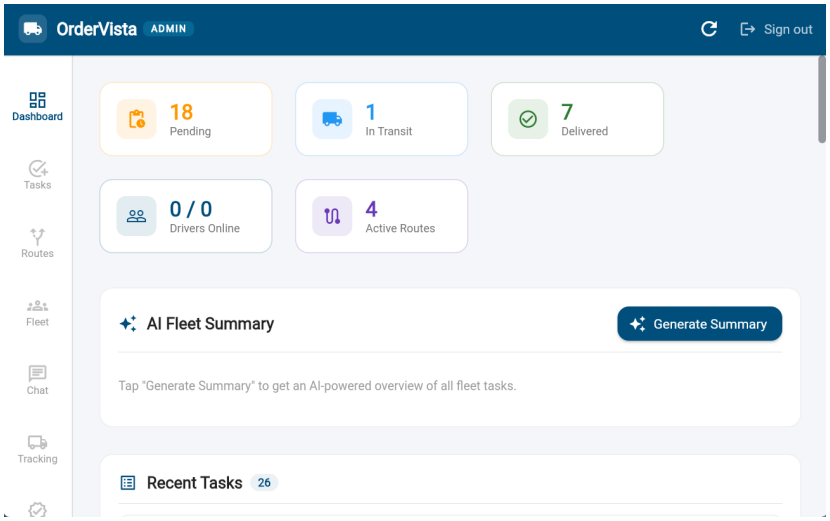


Figure D.3: Admin dashboard overview.

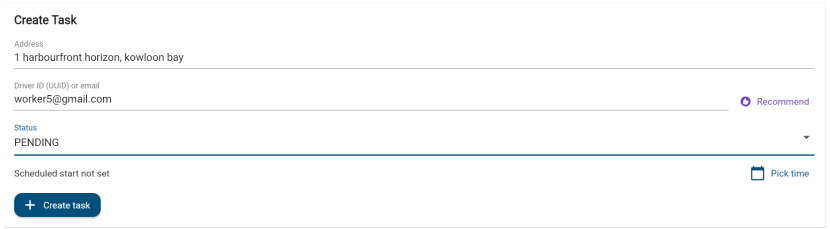


Figure D.4: Task creation form.

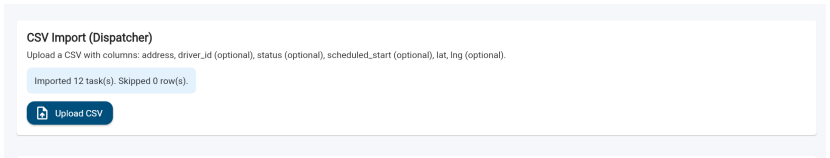


Figure D.5: CSV bulk import with column mapping.

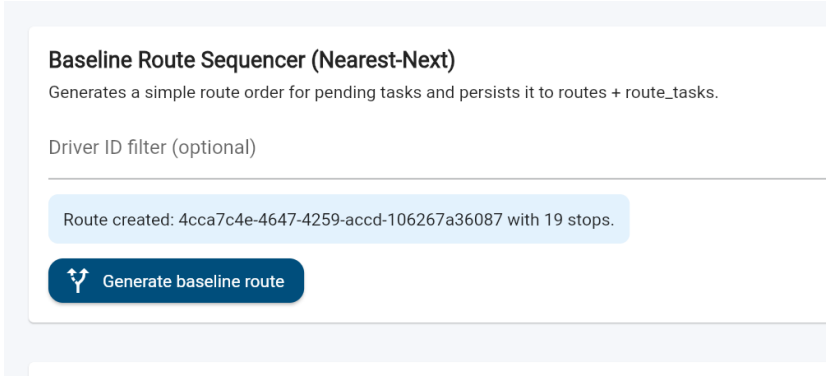


Figure D.6: Route sequencing with static map preview.

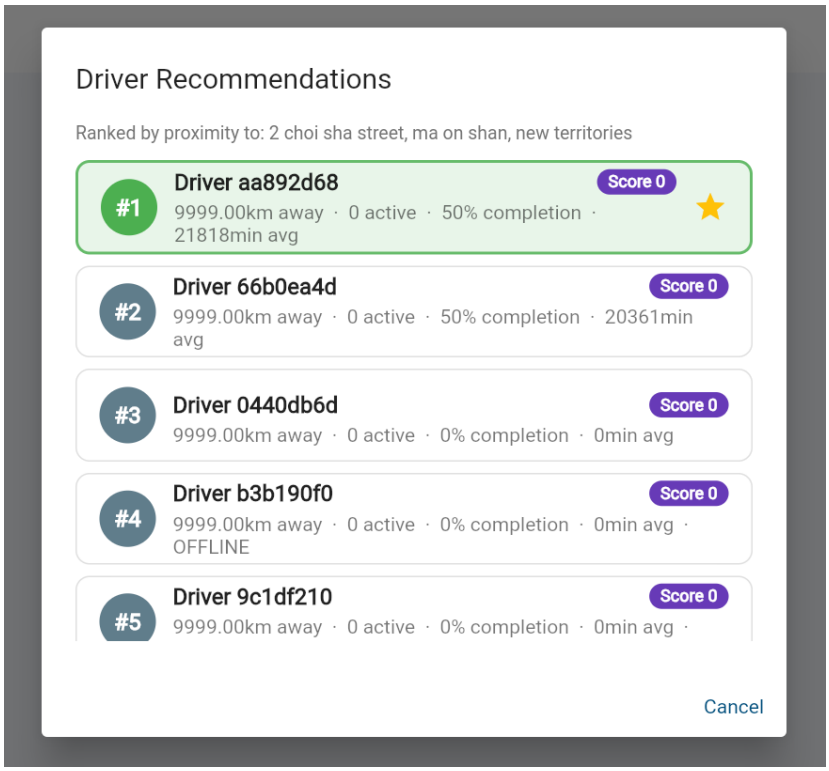


Figure D.7: Driver recommendation panel.

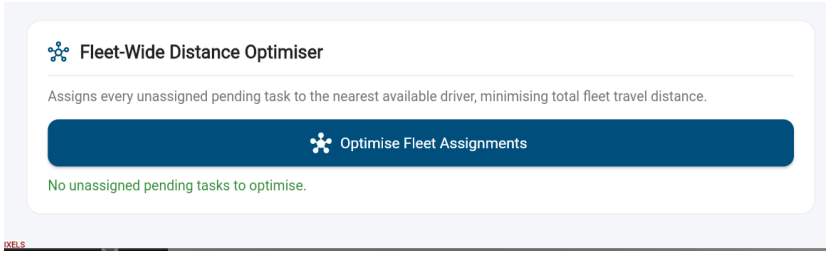


Figure D.8: Fleet and vehicle management.

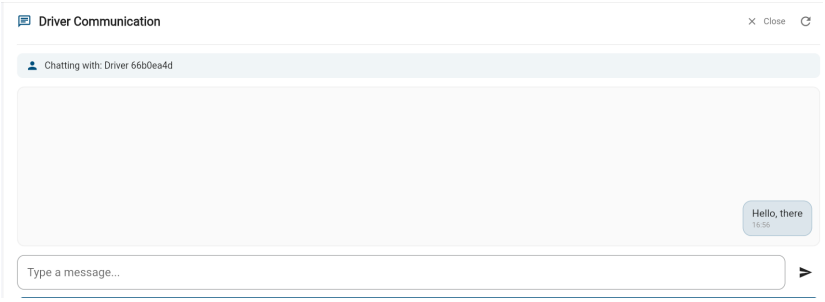


Figure D.9: Admin–driver real-time chat.

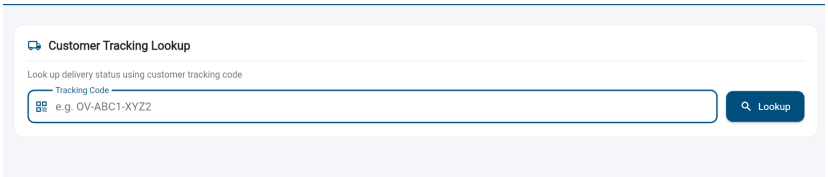


Figure D.10: Customer delivery tracking.

D.3 Driver Application

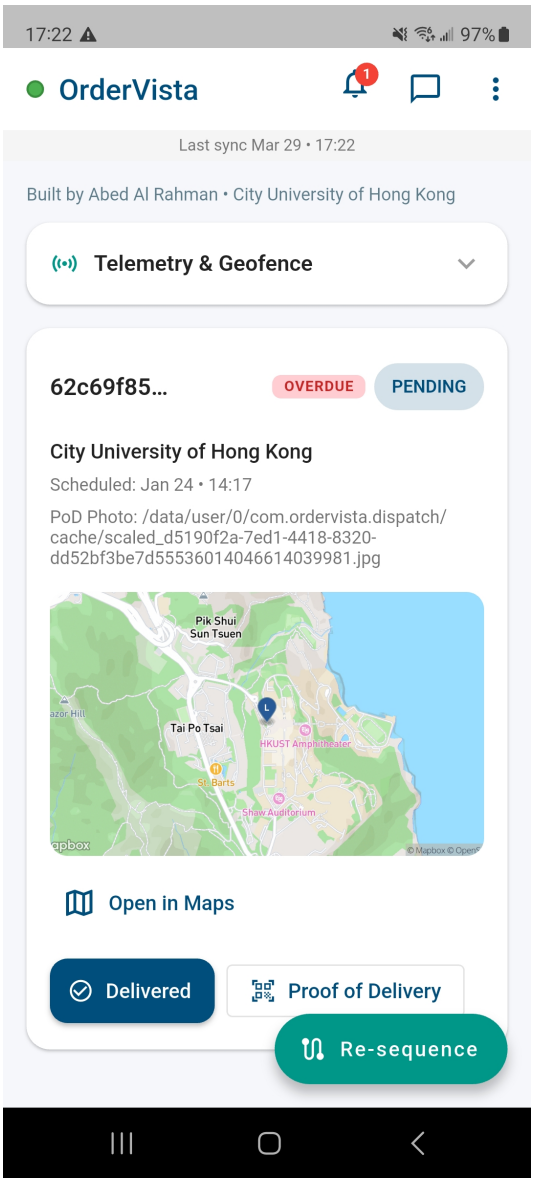


Figure D.11: Driver task list.

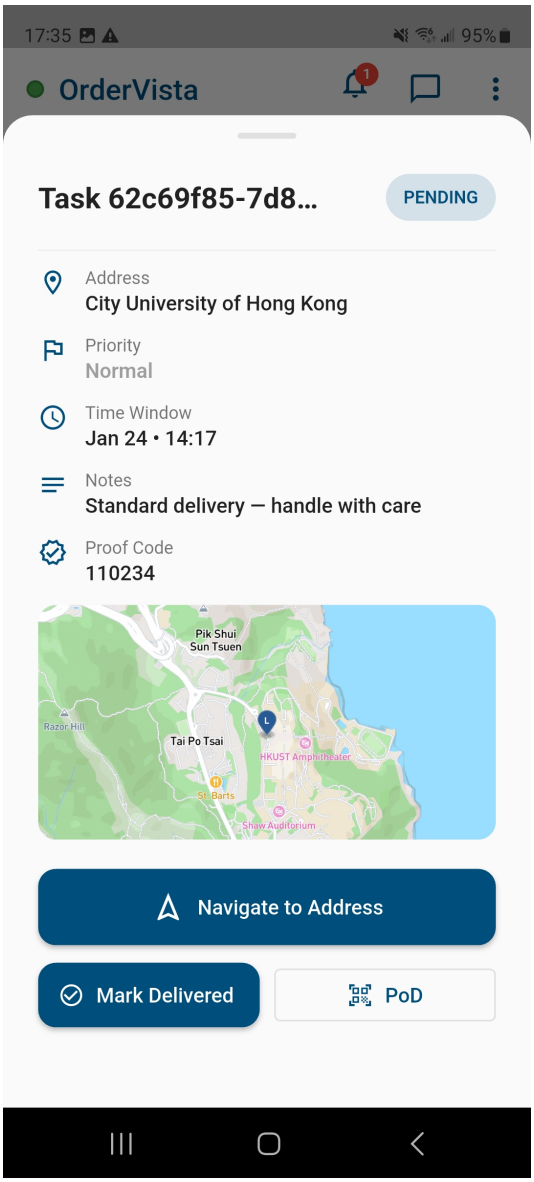


Figure D.12: Task detail and navigation.

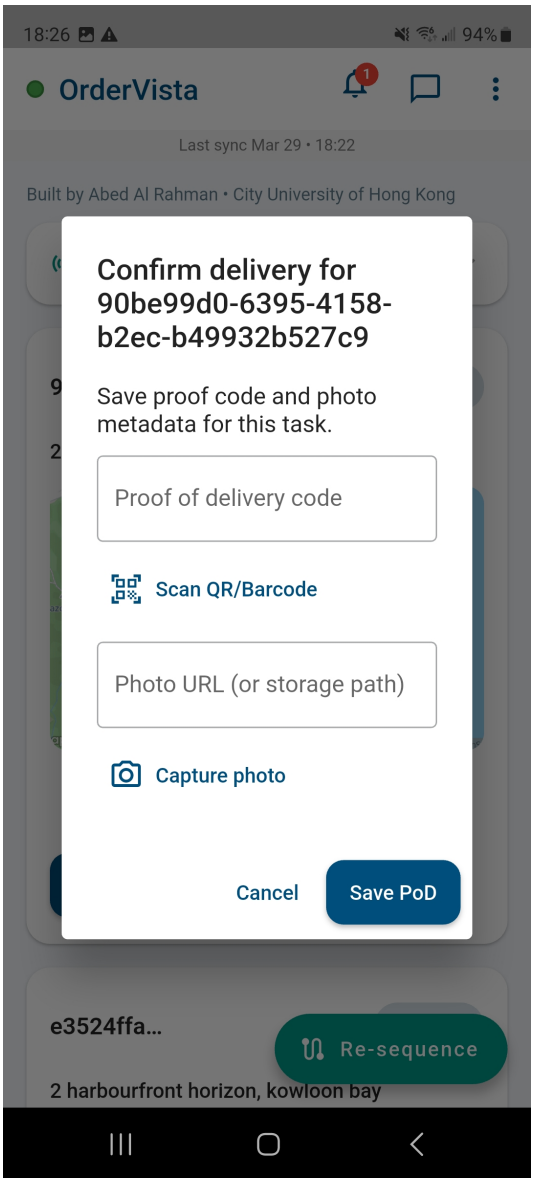


Figure D.13: Proof of delivery capture.

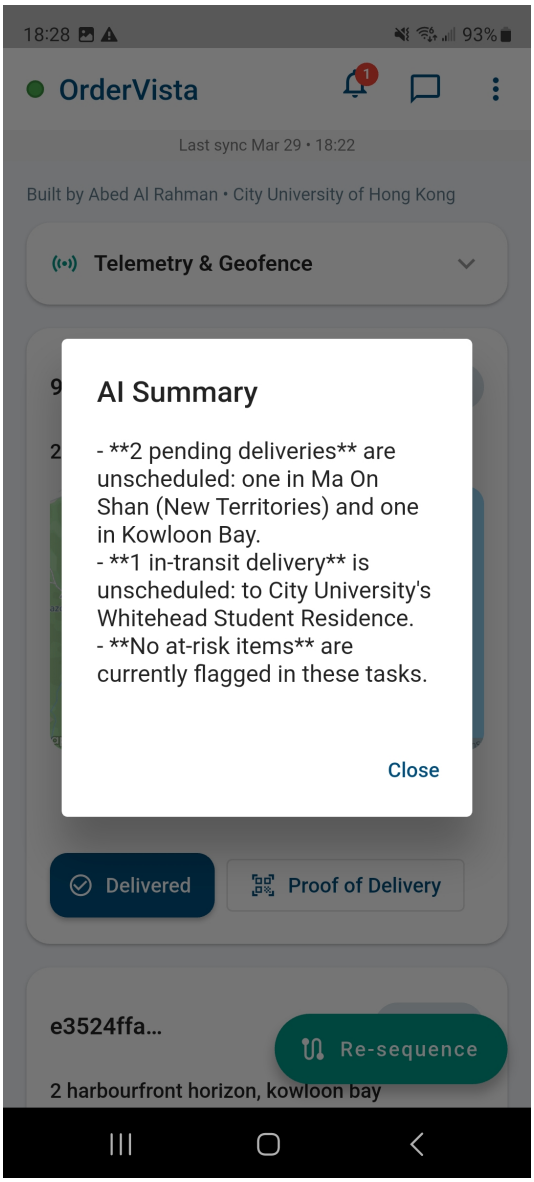


Figure D.14: AI assistant: summary and chat.

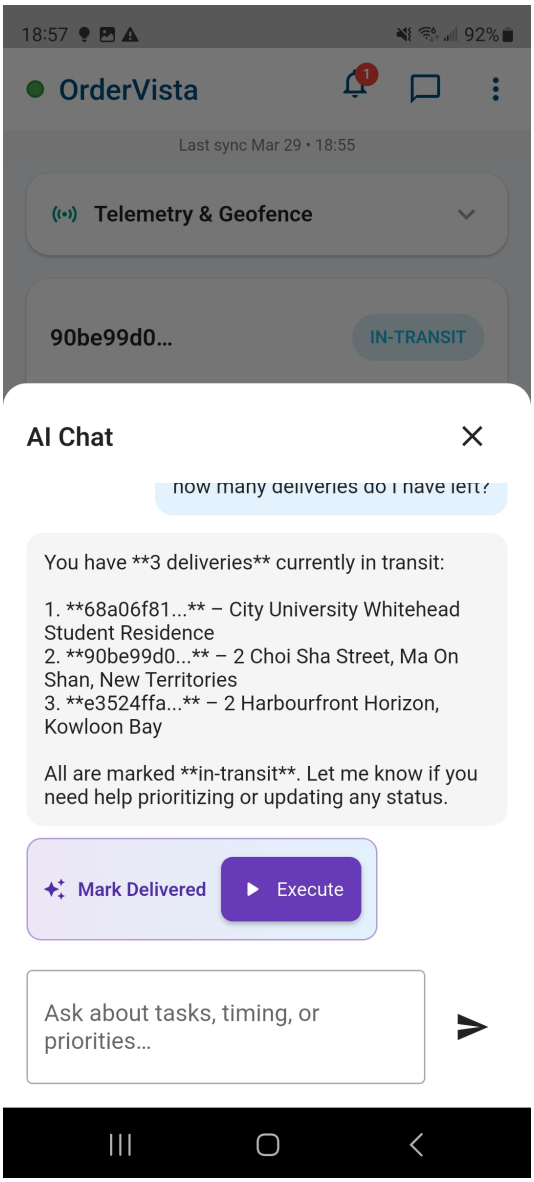


Figure D.15: Agentic AI action with one-tap execution.

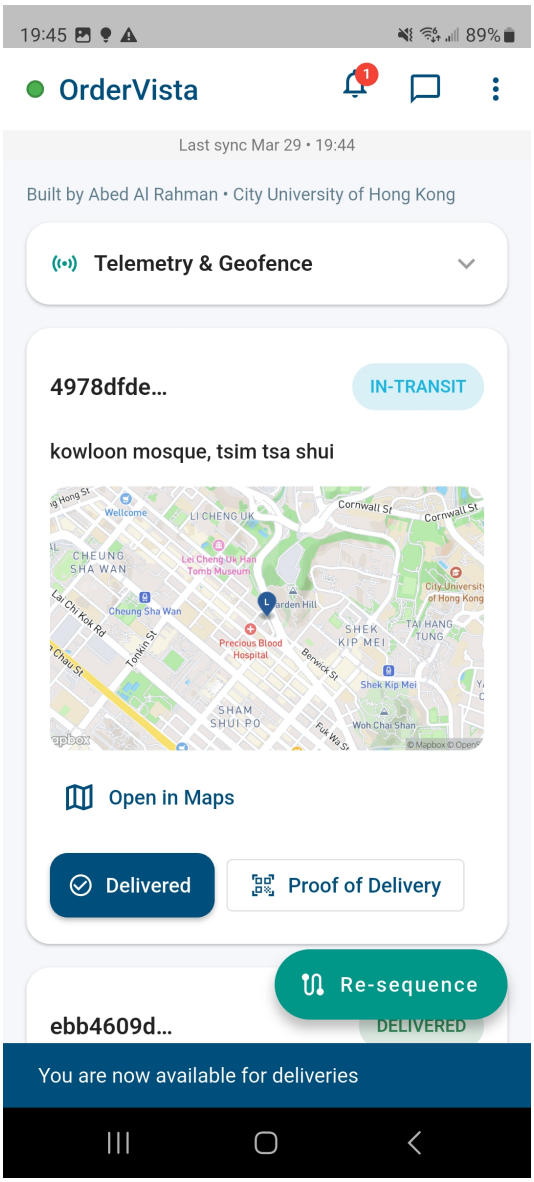


Figure D.16: Driver telemetry panel.

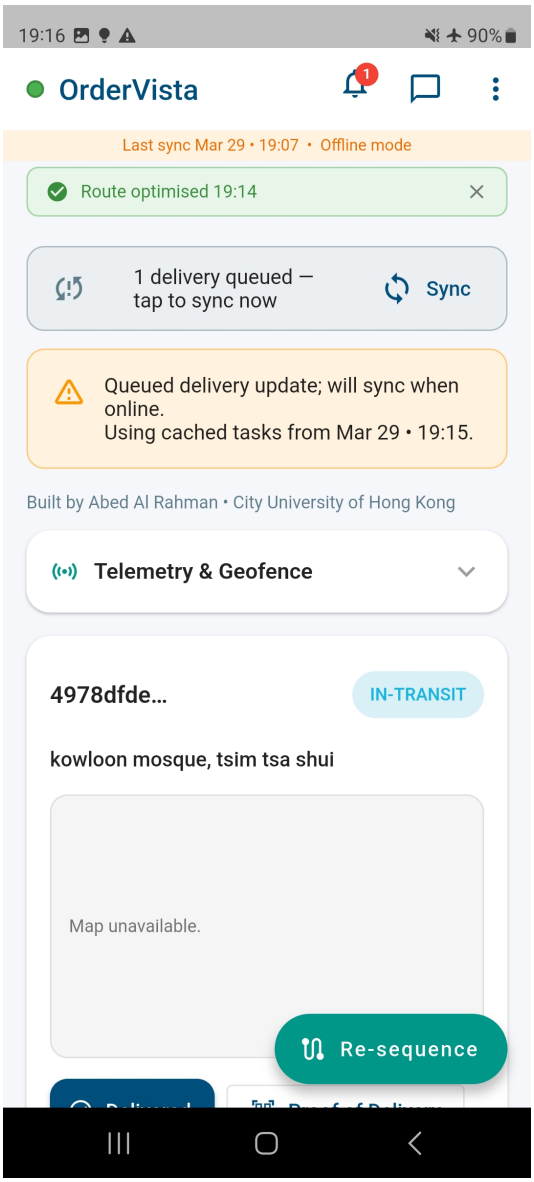


Figure D.17: Offline mode with sync queue.

Appendix E

Build and Deployment Evidence

This appendix documents the build configuration, dependency inventory, and deployment artefacts produced for the OrderVista project.

E.1 Build Environment

Table E.1: Build and development environment.

Component	Version / Detail
Flutter SDK	3.35.2 (stable channel)
Dart SDK	$\geq$ 3.0.0, $<$ 4.0.0
Android Gradle Plugin	8.9.1
Kotlin	2.1.0
Compile SDK	36 (Android 16)
Target SDK	35 (Android 15)
Min SDK	26 (Android 8.0 Oreo)
Java compatibility	1.8 (desugared with `desugar_jdk_libs 2.1.4`)
Development OS	Windows 10/11
IDE	Visual Studio Code with Flutter/Dart extensions
Version control	Git + GitHub (branch-per-feature)

E.2 Android Release Build

The signed release APK was produced using the following command:

```
flutter build apk --release
```

Table E.2: Release APK details.

Property	Value
Application ID	`com.ordervista.dispatch`
Version name	1.0.0
Version code	1
APK size	75.7 MB
Signing	Release keystore (SHA-256 signed)
ProGuard	Enabled (default rules)

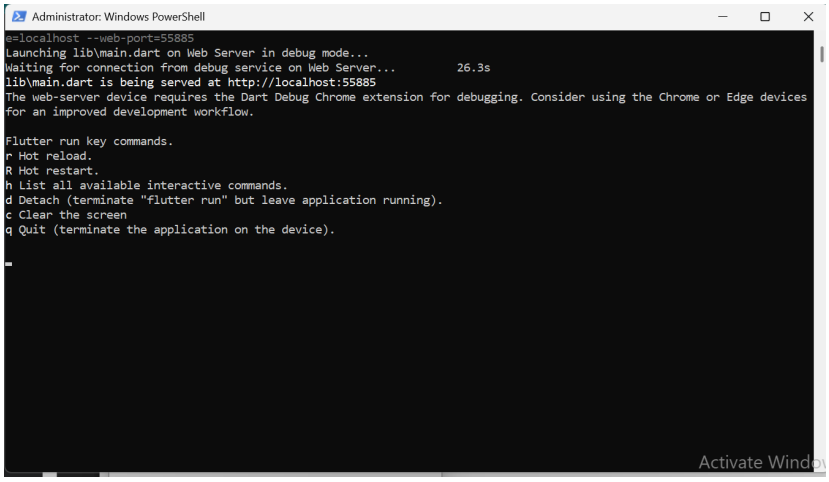


Figure E.1: Terminal output of a successful `flutter build apk -release`.

E.3 Backend Deployment

Table E.3: Supabase project configuration.

Setting	Value
Hosting	Supabase Cloud (free tier)
Region	Southeast Asia
PostgreSQL	15.x with PostGIS extension
Authentication	Supabase Auth (email/password)
Storage	Supabase Storage (PoD photos)
Realtime	Enabled for `chat_messages` and `tasks`
Migrations	3 SQL migration files (see Appendix C)

E.4 External Service Configuration

Table E.4: External API integrations.

Service	Purpose	Plan
Mapbox Geocoding API	Address → coordinates	Free tier
Mapbox Static Images	Route map previews	Free tier
OpenRouter API	LLM gateway	Pay-per-token
DeepSeek R1	AI model (via OpenRouter)	Free tier

E.5 Dependency Inventory

Table E.5 lists the principal Dart/Flutter packages used.

Table E.5: Key project dependencies.

Package	Version	Purpose
`supabase_flutter`	2.3.4	Supabase client (auth, DB, storage, realtime)
`drift`	2.18.0	SQLite ORM for offline cache
`sqlite3_flutter_libs`	0.5.22	Native SQLite bindings
`go_router`	12.1.3	Declarative routing with role guards
`provider`	6.1.5	State management (ChangeNotifier)
`geolocator`	11.0.0	GPS location services
`mobile_scanner`	4.0.1	Barcode and QR code scanning
`image_picker`	1.1.2	Camera photo capture
`csv`	6.0.0	CSV parsing for bulk import
`file_picker`	10.3.8	File selection dialog
`cached_network_image`	3.4.1	Image caching with placeholders
`google_fonts`	6.1.0	Typography
`flutter_animate`	4.5.0	Declarative animations
`intl`	0.20.2	Date/number formatting
`timeago`	3.7.1	Relative time display
`url_launcher`	6.3.1	External link/map navigation
`shared_preferences`	2.5.3	Key-value persistent storage
`json_path`	0.7.2	JSON querying for AI responses

E.6 Repository Structure

`ordervista/`

```
lib/
  main.dart              # Entry point
  app/                   # App shell, theme, router
  bootstrap/             # Initialisation, DI
  config/                # AppConfig, environment
  data/                  # Drift database, DAOs
  models/                # Domain models
  repositories/          # Repository pattern layer
  services/              # Supabase, Mapbox, AI services
  viewmodels/            # MVVM view-models
  screens/               # Admin portal screens
  pages/                 # Driver app pages
  components/            # Shared UI components
supabase/
  migrations/            # 3 SQL migration files
android/                 # Android build config
test/                    # Unit and widget tests
pubspec.yaml             # Dependencies
```

Appendix F

Extended Test Cases

This appendix provides the detailed test procedures that supplement the summary tables in Chapter 5. Each case specifies preconditions, step-by-step actions, and the expected versus observed outcome.

F.1 Admin Functional Tests

Table F.1: Extended admin test procedures.

ID	Test	Steps	Expected Result	Status
A1	Admin login	1. Open app. 2. Enter admin email and password. 3. Tap “Sign In”.	Dashboard loads; navigation shows admin menu items.	✓
A2	Create single task	1. Navigate to Tasks. 2. Tap “+ New Task”. 3. Enter address; wait for geocode. 4. Assign driver; set schedule. 5. Save.	Task appears in list with “pending” status and geocoded coordinates.	✓
A3	Edit existing task	1. Tap a pending task. 2. Change address and priority. 3. Save.	Updated fields persist on reload; `updated_at` timestamp refreshed.	✓
A4	Delete task	1. Tap a pending task. 2. Tap “Delete”. 3. Confirm.	Task removed from list and database.	✓

Continued on next page

ID	Test	Steps	Expected Result	Status
A5	CSV bulk import	1. Navigate to Import. 2. Select a CSV file (10+ rows). 3. Map columns. 4. Tap “Import”.	All rows geocoded and inserted; progress indicator completes; task count increases.	✓
A6	Generate optimised route	1. Select 5+ unassigned tasks. 2. Tap “Generate Route”. 3. Review map preview.	Route created with numbered stops in NN order; static map displays pins.	✓
A7	Assign driver to route	1. Open generated route. 2. Select a driver. 3. Confirm assignment.	Route status changes to “assigned”; driver sees tasks on their device.	✓
A8	Driver recommendation	1. Open a task or route. 2. Tap “Recommend Driver”.	Ranked list of drivers with availability, distance, completion rate, and composite score.	✓
A9	Review PoD	1. Navigate to a delivered task. 2. View proof panel.	Photo thumbnail and confirmation code visible; timestamp matches driver submission.	✓
A10	Send chat message	1. Open Chat. 2. Select a driver. 3. Type and send message.	Message appears in conversation; driver receives it in real time.	✓
A11	Broadcast message	1. Open Chat. 2. Tap “Broadcast”. 3. Type and send.	All drivers within the company see the broadcast.	✓

Continued on next page

ID	Test	Steps	Expected Result	Status
A12	AI fleet summary	1. Navigate to AI tab. 2. Tap “Generate Summary”.	DeepSeek returns a structured summary of today’s fleet activity.	✓

F.2 Driver Functional Tests

Table F.2: Extended driver test procedures.

ID	Test	Steps	Expected Result	Status
D1	Driver login	1. Open app. 2. Enter driver email and password. 3. Tap “Sign In”.	Task list loads; navigation shows driver menu items.	✓
D2	View assigned tasks	1. Navigate to task list after login.	All tasks assigned to the driver are visible with status badges.	✓
D3	View task details	1. Tap a task in the list.	Detail view shows address, notes, schedule, priority, and navigation button.	✓
D4	Navigate to task	1. Tap “Navigate” on task detail.	External maps app opens with task coordinates as destination.	✓
D5	Capture PoD photo	1. Tap “Complete Delivery”. 2. Take photo with camera. 3. Confirm upload.	Photo uploaded to Supabase Storage; task status changes to “delivered”.	✓

Continued on next page

ID	Test	Steps	Expected Result	Status
D6	Scan barcode/QR	1. Tap "Scan Code" on PoD screen. 2. Point camera at barcode.	Code captured and stored as `proof_code`.	✓
D7	AI task summary	1. Tap "AI Summary" on task list.	DeepSeek generates a natural-language summary of the day's deliveries.	✓
D8	AI conversational chat	1. Open AI chat. 2. Ask a question about remaining tasks.	Contextual answer returned referencing actual task data.	✓
D9	Agentic AI action	1. Ask AI to "notify the admin I'm running late". 2. Review the proposed action card.	Structured JSON action displayed with type, parameters, and "Execute" button.	✓
D10	Execute AI action	1. Tap "Execute" on the action card from D9.	Notification sent; confirmation displayed; action logged.	✓
D11	Driver chat	1. Open Chat. 2. View admin conversation. 3. Send a reply.	Message appears in thread; admin sees it in real time.	✓
D12	Toggle availability	1. Tap availability toggle.	Status updates to available/unavailable; admin portal reflects the change.	✓
D13	Mid-shift resequence	1. Complete a task. 2. Observe remaining task order.	Remaining tasks re-sequenced from current GPS position.	✓

Continued on next page

ID	Test	Steps	Expected Result	Status
D14	Offline task viewing	1. Disable network connectivity. 2. Open task list.	Cached tasks display with “offline” indicator; last-sync timestamp visible.	✓

F.3 Security Tests

Table F.3: Security test procedures.

ID	Test	Steps	Expected Result	Status
S1	RLS cross-tenant	1. Authenticate as Company A member. 2. Query `tasks` with Company B ID via Supabase client.	Empty result set; no data leakage.	✓
S2	Driver cannot delete tasks	1. Authenticate as driver. 2. Attempt `DELETE` on `tasks` table.	Operation denied by RLS policy.	✓
S3	Unauth. tracking	1. Call `lookup_delivery_by_tracking()` with valid code (no auth).	Returns task status and address only; no internal IDs exposed.	✓
S4	Invalid tracking code	1. Call `lookup_delivery_by_tracking()` with random string.	Returns null / empty; no error leakage.	✓

All test cases passed successfully on the Samsung Galaxy A54 physical device and Chrome desktop browser.